



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

TÍTULO DE LA MEMORIA/TESIS

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERE CIVIL EN COMPUTACIÓN

NOMBRE NOMBRE APELLIDO APELLIDO

PROFESOR GUÍA:
NOMBRE APELLIDO APELLIDO

PROFESORA CO-GUÍA:
NOMBRE APELLIDO APELLIDO

MIEMBROS DE LA COMISIÓN:
NOMBRE COMPLETO UNO
NOMBRE COMPLETO DOS
NOMBRE COMPLETO TRES

SANTIAGO DE CHILE
2026

RESUMEN DE LA MEMORIA PARA OPTAR AL
TÍTULO DE INGENIERE CIVIL EN COMPUTACIÓN
POR: NOMBRE NOMBRE APELLIDO APELLIDO
FECHA: 2026
PROF. GUIA: NOMBRE APELLIDO APELLIDO

TÍTULO DE LA MEMORIA/TESIS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Una dedicatoria especial para alguien especial.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Tabla de Contenido

1. Introducción	1
1.1. Problema abordado	1
1.2. Solución propuesta	2
1.3. Objetivos	3
1.4. Metodología	3
2. Trabajo relacionado	4
2.1. Técnicas tradicionales de IA en videojuegos	4
2.2. Enfoques de ajuste dinámico de dificultad	4
2.3. Tecnologías emergentes y aplicaciones recientes	5
2.4. Casos destacados de adaptación en la industria	5
2.5. Consideraciones de experiencia de usuario	7
2.6. Síntesis y brecha identificada	7
3. Diseño del videojuego	9
3.1. Género y mecánicas principales	9
3.2. Diseño y arquetipos de enemigos	10
3.3. Diseño de niveles	14
4. Sistema adaptativo basado en el comportamiento del jugador	17
5. Implementación del videojuego	18
5.1. Estadísticas del jugador	20
5.2. Recepción de daño y muerte	21
5.3. Componente de combate	22
5.4. Sistema de input	23
5.5. Ataque melee del jugador	24
5.6. proyectiles	25
5.7. Hechizos y consumibles del jugador	27
5.8. Esquiva	28

5.9. Fijado de objetivo (Target Lock)	29
5.10. Correr	30
5.11. Interactuar	31
5.12. Menú de pausa	31
5.13. Widgets del jugador	31
5.14. Enemigos	33
5.15. Jefe (BP_Slime)	40
5.16. Sistema de métricas y telemetría	46
5.17. Niveles	50
5.18. Widgets generales (MainMenu, WinnerScreen...)	52
6. Prueba de concepto y evaluación	53
7. Conclusión y trabajo futuro	54
Bibliografía	55
A. Anexo	56

Índice de Tablas

Tabla 1 Ataques del jefe agrupados según el rango respecto al jugador.	13
Tabla 2 Perfil de jugador al que apunta cada ataque del jefe.	14

Índice de Ilustraciones

Figura 1	Blueprint perteneciente al esqueleto normal	33
----------	---	----

Capítulo 1

Introducción

1.1. Problema abordado

El diseño de la inteligencia artificial en videojuegos ha avanzado notablemente en las últimas décadas, evolucionando desde simples máquinas de estados finitos hacia árboles de comportamiento y otros sistemas de navegación más sofisticados [1]. Sin embargo, muchos títulos continúan empleando enemigos con rutinas predefinidas y comportamientos relativamente previsibles, como ocurre en juegos como *Elden Ring*¹ o *Hollow Knight*², donde ciertos enemigos y jefes siguen patrones de ataque que los jugadores terminan memorizando tras varios intentos. En géneros particularmente exigentes en este aspecto, como los *souls-like*³ o los *roguelike*⁴, esta previsibilidad puede limitar la rejugabilidad, haciendo que la dificultad se perciba más como un ejercicio de memorización que como un desafío dinámico y en constante evolución.

Aunque existen técnicas para ajustar la dificultad, la mayoría de los acercamientos a la dificultad dinámica se limitan a modificar parámetros generales del juego -vida, daño o velocidad de los enemigos- en lugar de su comportamiento [2]. Son pocos los títulos que implementan sistemas donde los enemigos se adaptan activamente a las acciones específicas del jugador; uno de los ejemplos más citados es *Alien: Isolation*⁵, donde el antagonista aprende los patrones de escondite del jugador y ajusta sus rutas de patrullaje para mantener la tensión en cada encuentro.

¹Elden Ring es un videojuego de rol de acción en mundo abierto desarrollado por FromSoftware y publicado por Bandai Namco, ambientado en un universo creado junto a George R. R. Martin. Disponible en sitio web.

²Hollow Knight es un videojuego de acción y aventuras en 2D desarrollado por Team Cherry, ambientado en el reino subterráneo de Hallownest. Disponible en sitio web.

³Subgénero inspirado en la serie Dark Souls, caracterizado por un combate desafiante, penalizaciones por muerte y patrones de ataque memorizables en los jefes.

⁴Género de videojuegos caracterizado por mazmorras generadas aleatoriamente, *permadeath* (muerte permanente del personaje) y elementos de rol. Nombrado así por el juego Rogue (1980).

⁵Alien: Isolation es un videojuego de sigilo y supervivencia en primera persona desarrollado por Creative Assembly y publicado por SEGA, ambientado en el universo de la película Alien. Disponible en sitio web.

Este problema general -la falta de enemigos que aprendan del estilo de juego de cada jugador, en lugar de limitarse a ejecutar rutinas fijas o a escalar parámetros numéricos- es el que motiva este trabajo. Cuando un jugador logra memorizar los patrones de ataque o movimiento de un enemigo, el enfrentamiento pierde buena parte de su tensión y su sorpresa; si, en cambio, el enemigo identificara patrones en las acciones del jugador y ajustara su propia estrategia en consecuencia, un mismo enemigo podría ofrecer combates distintos según el estilo de juego de cada persona, generando experiencias más dinámicas, personalizadas y rejugables.

Este trabajo aborda dicho problema explorando la construcción de un enemigo final (un «jefe») adaptativo en Unreal Engine, motor ampliamente utilizado en la industria y reconocido por su robusto sistema de inteligencia artificial (Behaviour Trees, Blackboard) y su capacidad de prototipado rápido mediante Blueprints, lo que lo convierte en una plataforma adecuada para experimentar con este tipo de sistemas.

1.2. Solución propuesta

La solución desarrollada consiste en un sistema de inteligencia artificial adaptativa que permite a un enemigo jefe ajustar su comportamiento de combate en función de cómo juega cada jugador, implementado íntegramente con las herramientas nativas de Unreal Engine (Behaviour Trees, Blackboard y Blueprints, junto con algunos nodos personalizados en C++), sobre un juego del género *souls-like* construido especialmente para este trabajo.

A diferencia de otros enfoques de dificultad dinámica, que ajustan parámetros generales del juego (vida, daño, velocidad), la adaptación aquí propuesta opera sobre las probabilidades de selección de los distintos ataques del jefe: el repertorio de ataques se mantiene fijo, pero la frecuencia con que cada uno se elige se ajusta según un perfil de juego construido a partir del comportamiento del jugador frente a un conjunto de enemigos regulares previos al combate (capítulo 3) -su preferencia por el combate cuerpo a cuerpo o a distancia, su uso de la esquiva y sus hábitos de curación, entre otras señales.

Para poder evaluar el efecto de esta adaptación, se construyó también una versión de control del juego, idéntica a la versión adaptativa salvo en que los pesos de ataque del jefe permanecen fijos en sus valores base, sin incorporar el perfil de juego recolectado. La comparación entre ambas versiones (capítulo 6) permite aislar el efecto de la adaptación de otros factores del diseño del juego.

Cabe precisar que, durante la etapa de propuesta, se consideró además la posibilidad de extender esta adaptación al combate mismo -ajustando el comportamiento del jefe en tiempo real según las acciones del jugador durante el enfrentamiento- así como la de incorporar tecnologías externas de aprendizaje automático, como NVIDIA ACE. Ambas líneas quedaron fuera del alcance final de este trabajo, que se concentra en la adaptación previa al combate mediante las herramientas nativas del motor; se retoman como trabajo futuro en el capítulo 7.

1.3. Objetivos

1.3.1. Objetivo general

Diseñar e implementar un sistema de adaptación para un enemigo jefe en Unreal Engine, capaz de ajustar su comportamiento de combate a partir de un perfil construido sobre las acciones previas del jugador, con el fin de reducir la previsibilidad del enfrentamiento y evaluar su efecto sobre la experiencia de juego.

1.3.2. Objetivos específicos

1. Investigar y analizar enfoques existentes para el diseño de inteligencia artificial adaptativa en videojuegos.
2. Construir una versión base del juego con un jefe de comportamiento fijo, sin mecanismos adaptativos, que sirva como caso de control en las pruebas.
3. Diseñar e implementar un conjunto de enemigos regulares cuyo comportamiento frente al jugador permita construir un perfil de su estilo de juego.
4. Implementar, sobre la base anterior, un mecanismo que ajuste los pesos de selección de ataques del jefe a partir de dicho perfil antes del inicio del combate.
5. Validar el sistema mediante pruebas de jugabilidad, midiendo la capacidad de adaptación del jefe y la percepción de desafío por parte de los jugadores.
6. Comparar los resultados obtenidos entre la versión adaptativa y la versión de control, identificando ventajas, limitaciones y posibles mejoras.

1.4. Metodología

El desarrollo del trabajo siguió una metodología experimental de tipo control-tratamiento. A partir de una versión base del juego -con un jefe de comportamiento fijo, sin mecanismos de adaptación- se desarrolló una segunda versión, idéntica en el resto de sus sistemas, en la que el jefe ajusta los pesos de selección de sus ataques a partir de un perfil construido sobre el comportamiento del jugador frente a los enemigos regulares previos al combate (capítulos 3 y 4).

La evaluación del sistema contempla dos dimensiones. Desde una perspectiva técnica, se verifica la corrección del propio mecanismo de adaptación: a partir de los pesos de ataque volcados a archivo al iniciar cada combate (capítulo 4), se revisa que el perfil construido sobre el comportamiento del jugador se traduzca efectivamente en los ajustes esperados sobre dichos pesos. Desde la perspectiva de experiencia de usuario, se diseñaron pruebas de jugabilidad mediante un experimento controlado, en el que cada participante juega tanto la versión de control como la versión adaptativa en sesiones separadas, utilizando el Game Experience Questionnaire [3] para medir las dimensiones de desafío, inmersión, competencia y afecto, complementado con retroalimentación cualitativa sobre la percepción de predictibilidad de cada versión. El detalle de esta evaluación y sus resultados se presenta en el capítulo 6.

Capítulo 2

Trabajo relacionado

2.1. Técnicas tradicionales de IA en videojuegos

Actualmente, la mayoría de los videojuegos implementan la inteligencia de los enemigos mediante técnicas como los Behaviour Trees o las máquinas de estados finitos (FSM) [4]. Un Behaviour Tree es una estructura jerárquica que organiza decisiones y acciones mediante nodos que retornan éxito, fallo o ejecución en curso, lo que permite definir comportamientos de forma modular.

Estas metodologías permiten establecer patrones y reglas claras de reacción, ofreciendo control sobre el comportamiento de los enemigos. Sin embargo, carecen de capacidad de adaptación real: los enemigos suelen repetir las mismas rutinas y no aprenden de las acciones del jugador, lo que los hace predecibles tras algunos intentos y reduce tanto la rejugabilidad como el desafío.

2.2. Enfoques de ajuste dinámico de dificultad

En el ámbito de la investigación se han desarrollado diversos enfoques para el ajuste dinámico de dificultad [2], incluyendo métodos probabilísticos que optimizan el *engagement* del jugador mediante funciones matemáticas, sistemas basados en redes neuronales capaces de predecir estados emocionales del jugador con buena precisión, y técnicas de aprendizaje por refuerzo donde controladores adaptativos ajustan el comportamiento de los enemigos según el rendimiento del jugador, manteniendo experiencias equilibradas sin volverse predecibles. Aun así, estos casos siguen siendo excepcionales debido a diversos desafíos técnicos y de recursos, además de la limitada adopción por parte de la industria, y no representan una metodología estandarizada para lograr adaptación en tiempo real en los videojuegos modernos.

En esa misma línea, Zohaib [2] describe otros enfoques complementarios, como sistemas donde grupos de personajes no jugables forman comportamientos adaptativos de manera descentralizada, ajustando sus parámetros según la retroalimentación del jugador -por ejemplo, patrones de movimiento e interacciones-, así como técnicas de *scripting* dinámico

que modifican reglas de comportamiento en tiempo real con base en tasas de éxito y fracaso, permitiendo que los enemigos evolucionen sus tácticas durante el juego. Investigaciones más recientes, también recopiladas por el autor, han combinado algoritmos de búsqueda en árboles con redes neuronales para crear oponentes que adaptan su nivel de desafío según los recursos computacionales disponibles, demostrando que es posible correlacionar directamente la dificultad percibida con el tiempo de procesamiento asignado.

2.3. Tecnologías emergentes y aplicaciones recientes

En paralelo, tecnologías emergentes como NVIDIA ACE (*Avatar Cloud Engine*) han mostrado posibilidades interesantes de aprendizaje dinámico. Por ejemplo, en el juego MIR5⁶, un jefe conocido como *Asterion* utiliza esta tecnología para evaluar tácticas, habilidades y equipo del jugador en tiempo real, adaptando sus estrategias para mantener un desafío personalizado y constante. Aunque el enfoque principal de NVIDIA ACE ha sido mejorar la interacción con personajes no jugables mediante conversaciones naturales, su aplicación en MIR5 evidencia que es posible diseñar enemigos que aprenden y modifican su comportamiento según el jugador.

A pesar de estos avances, no existe una metodología ampliamente adoptada que permita implementar enemigos adaptativos de manera general en motores de juego comunes como Unreal Engine. La mayoría de las soluciones actuales dependen de tecnologías externas o están restringidas a casos específicos, y las técnicas más sofisticadas permanecen principalmente en el ámbito de la investigación, sin traducirse en herramientas accesibles para desarrolladores. Esto abre un espacio de investigación y desarrollo para crear sistemas de inteligencia artificial que no solo reaccionen, sino que aprendan y se ajusten al estilo de juego del usuario.

2.4. Casos destacados de adaptación en la industria

Más allá de Alien: Isolation y MIR5, existen otros casos notables en la industria que han explorado distintas formas de adaptación y personalización de la experiencia de juego.

2.4.1. Left 4 Dead y el AI Director

*Left 4 Dead*⁷ introdujo un sistema conocido como «AI Director», diseñado para crear experiencias dinámicas de terror y supervivencia cooperativa. A diferencia de los sistemas tradicionales, donde los enemigos aparecen en ubicaciones predefinidas, el AI Director analiza constantemente el estado del equipo de jugadores -incluyendo salud, munición, posición y nivel de estrés- para ajustar la intensidad del juego en tiempo real [5].

El sistema funciona manipulando varios parámetros: la frecuencia y tamaño de las hordas de zombis, la aparición de «infectados especiales» (enemigos con habilidades únicas),

⁶MIR5 es un MMORPG desarrollado por Wemade Next. Disponible en sitio web.

⁷Left 4 Dead es un videojuego cooperativo de supervivencia y terror desarrollado por Valve Corporation y publicado en 2008. El juego se centra en cuatro supervivientes que deben atravesar escenarios infestados de zombis, enfrentándose a hordas dinámicas controladas por el sistema AI Director. Disponible en sitio web.

la colocación de recursos como botiquines y munición, e incluso elementos ambientales como música y efectos de sonido. Cuando el equipo está funcionando bien, el Director incrementa la presión mediante encuentros más desafiantes; cuando los jugadores están al borde del colapso, reduce temporalmente la intensidad para permitir momentos de respiro y recuperación. Esta aproximación se basa en la teoría del «flow» de Csikszentmihalyi, que busca mantener a los jugadores en un estado óptimo entre el aburrimiento y la frustración.

Sin embargo, el AI Director presenta limitaciones importantes para el contexto de un combate contra un jefe individual: opera principalmente a nivel macro, ajustando parámetros globales del encuentro en lugar de modificar tácticas específicas de enemigos individuales. No aprende de las estrategias particulares de cada jugador -por ejemplo, si prefiere el combate a distancia o cuerpo a cuerpo- ni adapta patrones de ataque específicos; su enfoque se centra en el ritmo y la intensidad general de la experiencia, no en la creación de adversarios que evolucionen tácticamente según el estilo de juego individual.

2.4.2. Middle-earth: Shadow of Mordor y el Sistema Nemesis

El Sistema Nemesis, introducido en *Middle-earth: Shadow of Mordor*⁸ (2014) y expandido en su secuela *Shadow of War* (2017), representa un enfoque distinto hacia la personalización de enemigos. El sistema genera una jerarquía dinámica de capitanes orcos, cada uno con personalidad, fortalezas, debilidades y apariencia generadas proceduralmente.

Su característica más distintiva es la memoria persistente: cuando el jugador se enfrenta a un capitán, el resultado del encuentro tiene consecuencias permanentes. Si el jugador es derrotado, el capitán que lo mató asciende en rango, gana confianza y puede desarrollar nuevas habilidades o inmunidades; en encuentros posteriores, ese mismo capitán recordará la victoria anterior mediante diálogos contextuales y mostrará cicatrices o modificaciones físicas resultado del combate previo. Si el jugador logra herir pero no matar a un capitán que luego escapa, este puede regresar más tarde con vendajes, quemaduras o prótesis mecánicas que reflejan cómo fue herido, y potencialmente habiendo desarrollado resistencia a las tácticas que el jugador empleó.

No obstante, desde la perspectiva de adaptación táctica en combate, el Sistema Nemesis opera principalmente en la dimensión narrativa y estratégica más que en la táctica inmediata. Las adaptaciones de los capitanes son cambios de configuración entre encuentros -nuevas inmunidades, nuevas armas, nuevas habilidades- en lugar de ajustes dinámicos durante el combate mismo: un capitán no modifica sus patrones de ataque a mitad de pelea según observe que el jugador esquiva siempre hacia un mismo lado o abusa de cierta habilidad. Son evoluciones preprogramadas que se activan por condiciones específicas, en lugar de aprendizaje genuino de patrones de comportamiento del jugador.

⁸Middle-earth: Shadow of Mordor es un videojuego de acción y aventuras en mundo abierto desarrollado por Monolith Productions y publicado por Warner Bros. Interactive Entertainment en 2014, ambientado en el universo de El Señor de los Anillos. El juego presenta el Sistema Nemesis, que genera enemigos dinámicos con memoria persistente de sus encuentros con el jugador. Disponible en sitio web.

2.4.3. Limitaciones comunes y espacio de oportunidad

Ambos sistemas, aunque innovadores en sus respectivos dominios, comparten una limitación de fondo: ninguno implementa adaptación táctica en tiempo real basada en el análisis de patrones específicos del estilo de combate del jugador. *Left 4 Dead* ajusta la experiencia global, pero no personaliza el comportamiento de enemigos individuales; *Shadow of Mordor* genera evolución narrativa y estratégica entre encuentros, pero no adaptación táctica durante los combates mismos.

Esto evidencia un espacio de oportunidad: enemigos -en particular, jefes- capaces de identificar y responder a las preferencias tácticas del jugador (patrones de ataque, *timing* de esquivas, uso de habilidades específicas, posicionamiento), ajustando su comportamiento de forma dinámica antes o durante el combate. Una aproximación así complementaría los enfoques existentes al enfocarse en la adaptación a nivel micro-táctico, donde cada encuentro individual se vuelve un desafío personalizado según las decisiones momento a momento del jugador.

2.5. Consideraciones de experiencia de usuario

Más allá de la implementación técnica de los sistemas adaptativos, es necesario considerar cómo estos impactan la experiencia del usuario. Pinelle et al. [6] desarrollaron un conjunto de heurísticas para evaluar la usabilidad en videojuegos, derivadas del análisis de 108 reseñas profesionales que cubrieron seis géneros principales. Su trabajo identificó doce categorías de problemas de usabilidad comunes, entre las cuales destacan: respuestas impredecibles a las acciones del usuario, falta de información sobre el estado del juego, controles difíciles de manejar y representaciones visuales difíciles de interpretar.

Estas heurísticas son particularmente relevantes para el desarrollo de enemigos adaptativos, ya que varias se relacionan directamente con cómo el jugador percibe y entiende el comportamiento de la IA. Por ejemplo, la heurística de proporcionar respuestas consistentes a las acciones del usuario establece que los enemigos deben comportarse de manera predecible y apropiada para la situación, lo que plantea una tensión de fondo para este trabajo: ¿cómo lograr que un enemigo sea adaptativo sin volverse impredecible o frustrante? Un sistema de este tipo debe equilibrar la adaptación dinámica con la consistencia percibida, de manera que el jugador pueda entender las «reglas» del comportamiento del enemigo incluso cuando este evolucione.

2.6. Síntesis y brecha identificada

La revisión de la literatura muestra que, aunque existen enfoques tradicionales y avances recientes en IA de videojuegos, ninguno ofrece una solución general ni fácilmente integrable para lograr adaptación táctica en tiempo real en enemigos individuales dentro de motores como Unreal Engine. Las técnicas clásicas no aprenden del jugador, los métodos de investigación suelen requerir recursos elevados o no están pensados para su aplicación práctica en juegos comerciales, y los casos industriales corresponden a soluciones específicas, difíciles de replicar.

Esta situación evidencia una brecha concreta: la falta de un mecanismo que permita que un enemigo -en particular, un jefe- registre el comportamiento del jugador y ajuste sus patrones de combate de manera dinámica, implementado con las herramientas nativas de un motor de uso común. Sobre esa brecha se construye el sistema descrito en los capítulos siguientes.

Capítulo 3

Diseño del videojuego

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

3.1. Género y mecánicas principales

El juego corresponde a un *action-RPG* en tercera persona del género *souls-like*, mencionado ya en el capítulo 1: se compone de un combate en tiempo real sostenido por estadísticas de vida, stamina y maná, y de una dificultad pensada para premiar la lectura atenta de cada enemigo por sobre la repetición de un mismo botón. Esta elección de género combina un gusto personal con una razón de fondo que sí es relevante para este trabajo: es precisamente en este tipo de juegos donde la previsibilidad de los enemigos se vuelve más notoria -el jugador aprende a memorizar patrones de ataque tras unos pocos intentos-, por lo que resulta el escenario más exigente para poner a prueba un enemigo que efectivamente se adapte a quien lo enfrenta.

Sobre esa base, el juego se construyó alrededor de un pequeño conjunto de mecánicas que definen lo que el jugador puede hacer en cada encuentro:

- **Desplazamiento:** el jugador se mueve libremente en cualquier dirección en tercera persona.
- **Combate cuerpo a cuerpo:** el jugador ataca con una secuencia de golpes consecutivos, pudiendo encadenarlos en un combo si continúa atacando a tiempo.

- **Esquiva:** el jugador puede rodar para evadir un ataque, quedando invulnerable durante toda la duración del movimiento; es la principal herramienta defensiva del juego.
- **Fijado de objetivo:** el jugador puede fijar la cámara sobre un enemigo específico, orientándose automáticamente hacia él durante el combate, lo que le permite desplazarse lateralmente a su alrededor sin perderlo de vista.
- **Equipamiento alternable:** mediante una misma acción, el jugador puede alternar entre un hechizo de proyectil de fuego -ofensivo, a distancia, con costo de maná- y una poción de vida -defensiva, de uso limitado-, ofreciendo una vía alternativa al combate cuerpo a cuerpo puro.
- **Carrera:** el jugador puede desplazarse más rápido de lo normal, útil para ganar o mantener distancia frente a un enemigo.

Estas acciones se sostienen sobre los recursos que limitan su uso: la vida, que termina el encuentro si llega a cero; la stamina, que se consume al esquivar o correr; el maná, que se consume al lanzar el hechizo de fuego; y una cantidad limitada de pociones de vida, que es lo que efectivamente acota cuándo el jugador puede curarse. Esta limitación de recursos obliga al jugador a tomar decisiones constantes -cuándo esquivar en lugar de atacar, cuándo curarse en lugar de seguir presionando, cuándo alejarse en lugar de arriesgar un golpe-, en lugar de simplemente repetir una única acción óptima.

Esta variedad de decisiones es, además, lo que hace posible el resto del diseño descrito en este capítulo: si el juego solo permitiera atacar y nada más, todos los jugadores jugarían igual y no habría comportamiento que perfilar. No todas las mecánicas cumplen ese rol por igual: el fijado de objetivo y la carrera responden más bien a la convención del género -se esperan en un juego de este tipo y facilitan el manejo de la cámara y el posicionamiento- sin que el sistema observe especialmente cómo se usan. Son, en cambio, la preferencia por el cuerpo a cuerpo o el hechizo a distancia, el uso más o menos frecuente de la esquiva, y el momento en que se decide curar, las mecánicas que los arquetipos de enemigos descritos a continuación están pensados para poner a prueba.

3.2. Diseño y arquetipos de enemigos

El diseño de los enemigos no buscó únicamente escalar la dificultad de forma progresiva, sino representar distintos arquetipos de combate, cada uno pensado para poner a prueba una forma distinta de jugar. En lugar de variar solo la vida o el daño de un mismo tipo de enemigo, se optó por diferenciar a cada uno por su patrón de comportamiento, de manera que el jugador deba ajustar su forma de jugar -y no solo su nivel de atención- al enfrentar a cada arquetipo.

Esta decisión responde a dos objetivos que se buscó conciliar. El primero es puramente de diseño de juego: un elenco pequeño de enemigos claramente diferenciados resulta más legible para el jugador que uno numeroso pero homogéneo, ya que cada encuentro comunica de inmediato qué tipo de respuesta espera de quien lo enfrenta. El segundo objetivo, propio de este proyecto, es que esa misma diferenciación permite que el comportamiento del

jugador frente a cada arquetipo sea informativo: si los enemigos fuesen variaciones menores entre sí, la forma en que el jugador los enfrenta no distinguiría con claridad un estilo de juego de otro. Por ello, los tres enemigos «regulares» del juego se diseñaron para ser, en la práctica, situaciones de prueba distintas entre sí, mientras que el jefe se diseñó como un antagonista capaz de responder a los resultados de esas pruebas. El detalle de cómo se llega a esa progresión a lo largo de los niveles del juego se aborda en la siguiente sección.

El primer arquetipo, el esqueleto normal, corresponde al enemigo más básico del juego: un enemigo cuerpo a cuerpo que persigue al jugador y cuyo único ataque es un golpe directo de espada, sin variaciones ni segundas intenciones. Su rol es introducir el ciclo de combate fundamental -acercarse, esquivar y golpear- sin agregar elementos adicionales que puedan confundir al jugador mientras aún está aprendiendo los controles. Por este motivo es el único enemigo presente durante el tutorial. Su segunda aparición, ya en el nivel previo al combate y junto a los demás arquetipos, no busca representar un desafío en sí mismo, sino servir de punto de comparación: al ser un enemigo «neutro», la forma en que el jugador lo enfrenta funciona como una referencia base respecto a la cual contrastar su comportamiento frente a los dos arquetipos siguientes, ambos más especializados.

El segundo arquetipo, el esqueleto mago, se diseñó como contraparte directa del anterior: en lugar de perseguir siempre al jugador hasta el cuerpo a cuerpo, su acercamiento a distancia es configurable mediante una variable propia, que permite optar entre seguir aproximándose -hasta una distancia mayor a la que usa el esqueleto normal para iniciar su ataque- o permanecer directamente en su posición lanzando proyectiles una vez detecta al jugador por primera vez. En su configuración actual privilegia esta segunda opción, recurriendo a un golpe cuerpo a cuerpo con su cayado únicamente cuando es el propio jugador quien decide acortar el espacio entre ambos. Su propósito es introducir una decisión táctica explícita que el esqueleto normal no plantea -acercarse para forzar el combate cuerpo a cuerpo contra un enemigo que de otro modo se mantiene a distancia, o permanecer lejos y lidiar con sus ataques a rango-, y con ello exponer una primera diferencia de fondo entre estilos de juego: quienes prefieren resolver los encuentros de forma agresiva y cercana, frente a quienes prefieren un acercamiento más cauteloso y a distancia. Esta es, de los tres arquetipos regulares, la diferencia de comportamiento más directa de observar. Concretamente, el mago alterna entre dos ataques:

- **Ataque a distancia:** lanza un proyectil hacia el jugador desde lejos, sin necesidad de acercarse.
- **Ataque cuerpo a cuerpo:** golpea con su cayado cuando el jugador se encuentra a corta distancia, recurriendo a él solo en ese caso.

El tercer arquetipo, el esqueleto caballero, se diseñó con un propósito distinto a los dos anteriores: a diferencia del esqueleto normal y del mago, cuyos ataques se resuelven con relativamente poca preparación visible, los dos ataques del caballero están marcadamente

teleografiados, es decir, tienen una preparación larga y claramente reconocible antes de conectar:

- **Avance (dash):** el caballero cubre distancia rápidamente para golpear, obligando al jugador a decidir hacia qué lado o dirección esquivar.
- **Golpe de área:** de preparación aún más larga que el avance, castiga con especial fuerza a quien no logra esquivarlo a tiempo.

Su rol no es presionar al jugador con velocidad o cantidad de golpes, sino enseñarle a identificar una señal de ataque evidente y a esquivarla con el timing correcto, en lugar de reaccionar de forma genérica ante cualquier ataque. Pero ese rol no es solo pedagógico: junto con enseñarle al jugador a leer este tipo de señales, el caballero es también el arquetipo a través del cual el juego observa cómo responde efectivamente el jugador frente a ellas -si esquivo el dash hacia un lado o hacia atrás, y si logra o no esquivar el ataque teleografiado a tiempo-, comportamiento que pasa a formar parte del perfil de juego utilizado más adelante para adaptar al jefe (capítulo 4). Al ser el primer enemigo del juego que exige este tipo de lectura, su rol dentro de la progresión es cerrar la preparación del jugador justo antes del enfrentamiento contra el jefe, cuyos ataques son considerablemente más numerosos y variados.

Tomados en conjunto, estos tres arquetipos no se diseñaron para ser superados de forma aislada, sino como un pequeño catálogo de situaciones de combate -ritmo básico, gestión de distancia y lectura de ataques teleografiados- que, sumadas, dan una imagen razonablemente completa de cómo juega quien los enfrenta. Por eso, antes que pensarlos como tres enemigos sueltos, conviene entenderlos como las tres «preguntas» que el juego le hace al jugador antes de llegar al jefe.

Con respecto al jefe, este ocupa un lugar distinto dentro de esta lógica: no es una prueba más que se suma a las anteriores, sino el punto donde todas ellas convergen. Mientras los tres arquetipos regulares plantean preguntas por separado -ritmo, distancia, lectura de ataques-, el jefe es el primer enemigo que responde a las respuestas que el jugador ya dio. Por eso se descartó deliberadamente la alternativa más común en el género -múltiples jefes, o un mismo jefe dividido en fases fijas-, ya que en ese esquema la dificultad se resuelve eligiendo qué enemigo o qué fase enfrenta cada jugador, una variable externa a su propio desempeño. Aquí, en cambio, es un único antagonista versátil el que cambia, y lo hace en función de cómo se comportó quien lo enfrenta. A diferencia de los demás enemigos, que reutilizan assets ya existentes, el jefe se construyó como un personaje propio, lo que permitió darle distintas formas y diseñar cada uno de sus ataques a la medida de las necesidades del combate, en lugar de adaptarse a las limitaciones de un modelo externo: el mismo jefe puede presionar la distancia, multiplicar los ataques teleografiados o variar su agresividad según el perfil construido a partir de los arquetipos anteriores. Este diseño es

lo que hace posible el sistema adaptativo del capítulo 4 -el jefe no es un desafío fijo al final de la progresión, sino su consecuencia directa.

Concretamente, el jefe dispone de nueve ataques distintos, agrupados en tres secuencias según el rango al que se encuentre el jugador en el momento de atacar. La Tabla 1 resume esta relación.

Rango	Ataques disponibles
Lejano	Charco, persecución, proyectil en línea recta, proyectil homing
Medio	Básico, salto, giro
Cercano	Básico, espinas, muro

Tabla 1: Ataques del jefe agrupados según el rango respecto al jugador.

A grandes rasgos, cada uno de estos ataques se comporta de la siguiente forma:

- **Básico:** un ataque cuerpo a cuerpo en el que el jefe deforma su cuerpo hacia adelante para golpear.
- **Espinas:** el jefe saca espinas alrededor de sí mismo, golpeando en área a quien se encuentre cerca.
- **Muro:** el jefe se transforma en un muro rectangular, se estira hacia arriba y cae hacia adelante, aplastando al jugador frente a él.
- **Salto:** el jefe salta acercándose a la posición del jugador y golpea en área al aterrizar.
- **Giro:** el jefe se contrae brevemente y luego extiende su cuerpo como un látigo, describiendo un barrido lateral frente a él.
- **Charco:** el jefe se transforma en un charco líquido y avanza hacia el jugador, para luego, tras un tiempo, resurgir desde el suelo golpeando y repeliendo al jugador si este se encuentra cerca.
- **Persecución:** el jefe se desplaza hacia el jugador durante un máximo de algunos segundos o hasta entrar en un rango de ataque determinado; no causa daño por sí misma, sino que sirve para cerrar distancia desde un rango medio-lejano.
- **Proyectil en línea recta:** el jefe dispara un abanico de proyectiles que avanzan en línea recta, sin seguir al jugador.
- **Proyectil homing:** el jefe dispara una cantidad menor de proyectiles, pero que persiguen activamente al jugador.

Cada uno de estos ataques se diseñó, además, pensando en un perfil de jugador distinto al cual poner a prueba, tal como resume la Tabla 2.

Ataque	Perfil de jugador que pone a prueba
Básico	Jugador estático, que permanece quieto frente al jefe.
Espinas	Jugador agresivo, que se mantiene cerca del jefe.
Muro	Ninguno en particular; aporta variedad táctica a corta distancia.
Giro	Jugador que se desplaza lateralmente en rango medio.
Salto	Jugador que mantiene distancia media o esquiva hacia atrás.
Charco	Jugador que huye constantemente o abusa del <i>kiteo</i> .
Persecución	Ninguno en particular; es solo movilidad hacia el jugador.
Proyectil en línea recta	Jugador que permanece quieto a distancia lejana.
Proyectil homing	Jugador que se mueve mucho estando a distancia lejana.

Tabla 2: Perfil de jugador al que apunta cada ataque del jefe.

Dentro de cada rango, el ataque concreto se elige mediante un sorteo aleatorio ponderado, y son justamente esos pesos -no la elección del rango, que depende solo de la distancia- los que el sistema adaptativo del capítulo 4 ajusta según el comportamiento previo del jugador. Cabe mencionar que todos los ataques parten con el mismo peso, es decir, la misma probabilidad de ser elegidos dentro de su rango, antes de que el sistema adaptativo introduzca cualquier ajuste. El detalle de cada ataque y del mecanismo de selección se documenta en el capítulo 5.

El detalle técnico de la implementación de cada uno de estos enemigos -estructura, comportamiento e inteligencia artificial- se documenta en el capítulo 5.

3.3. Diseño de niveles

Los niveles se diseñaron como una progresión que ordena, en el tiempo y el espacio, los encuentros descritos en la sección anterior: primero se introduce al jugador en un entorno de bajo riesgo, luego se le enfrenta a los tres arquetipos regulares en conjunto, y finalmente se le lleva al combate contra el jefe. La ambientación de mazmorra que comparten los tres niveles responde a una decisión de producción -aprovechar assets disponibles de Fab (Unreal Engine Marketplace)- y no condiciona las decisiones de diseño que se describen a continuación.

3.3.1. Nivel de tutorial

El primer nivel se diseñó como un entorno de bajo riesgo, cuyo objetivo es que el jugador aprenda los controles y mecánicas básicas (desplazamiento, esquiva, fijado de objetivo, ataque) antes de que estas decisiones tengan consecuencias relevantes. Por esto, el único enemigo que hay en el nivel es el esqueleto normal -el arquetipo más simple-, precedido por un maniquí sin capacidad de respuesta, de modo que la dificultad real del combate se introduce de forma gradual.

El nivel se organiza en una secuencia de salas, cada una dedicada a introducir una mecánica nueva mediante un mensaje emergente, seguida de inmediato por una oportunidad de practicarla antes de poder continuar.

La sala inicial, donde aparece el jugador, enseña el desplazamiento y presenta al maniquí: al acercarse, un mensaje indica que el clic izquierdo ataca, y que presionarlo varias veces seguidas encadena los golpes en un combo. Recién cuando el jugador logra bajarle la vida al maniquí se abre la puerta de salida.

La segunda sala es notablemente más larga que la primera. En su recorrido se enseña a correr y, ya cerca del primer esqueleto normal que el jugador encuentra, a esquivar, aclarando que la esquiva otorga invulnerabilidad. Derrotar a este enemigo abre la puerta hacia la tercera sala.

La tercera sala está dividida por un abismo con espinas en el fondo, cruzado por un puente que comienza levantado, con un maniquí ubicado al otro lado. Al entrar, se enseña a fijar el objetivo y a lanzar una bola de fuego; la intención es que el jugador fije al maniquí del otro lado del abismo y le baje la vida a distancia, ya que el puente permanece levantado y el abismo impide cruzar de otra forma. Al lograrlo, el puente baja y habilita el paso.

Al otro lado, unas flechas señalan un agujero en el suelo por el que el jugador debe dejarse caer para continuar. Tras la caída, el jugador aterriza sobre un campo de espinas en la sala siguiente, donde se enseña a alternar entre el hechizo de fuego y la poción de vida, pidiéndole específicamente cambiar a la poción y curarse.

El nivel termina poco después, en un pasillo final sin más obstáculos.

3.3.2. Nivel previo al jefe

El segundo nivel reúne, por primera y única vez, a los tres arquetipos de enemigo regulares -esqueleto normal, mago y caballero- descritos en la sección anterior. A diferencia del tutorial, aquí el jugador debe lidiar con los tres ejes de comportamiento -elección de rango, manejo de la esquiva, lectura de ataques telegrafados- de forma simultánea y bajo la presión de varios enemigos a la vez, lo que lo convierte en el último peldaño de dificultad antes del jefe.

El nivel se organiza en torno a una sala amplia central, desde la cual se accede a dos zonas dedicadas -una por cada uno de los arquetipos especializados- antes de poder llegar a la puerta del jefe.

El jugador comienza en una sala vacía y, al avanzar por un primer tramo de pasillos, se encuentra de inmediato con el primer esqueleto normal, que patrulla la zona. Al final de estos pasillos se abre la sala amplia mencionada, donde patrullan dos esqueletos normales adicionales.

Al frente de esta sala, en el extremo opuesto a la entrada, se encuentra la puerta de acceso al jefe, bloqueada por dos puertas metálicas dispuestas una tras otra. Abrir ambas es la condición para avanzar, y cada una se desbloquea completando una de las dos zonas laterales que se describen a continuación. Además de la entrada por los pasillos y de esta puerta bloqueada, la sala amplia tiene otras dos salidas -una hacia cada zona-, junto con dos puertas metálicas adicionales que funcionan únicamente como atajos de regreso.

La salida izquierda lleva, mediante una escalera, a un segundo piso que corresponde a la zona de enemigos tanque, es decir, a los esqueletos caballero. Tras avanzar un poco se encuentra el primer caballero y, más adelante en la misma sala, el segundo. Al final de la zona, una palanca abre a la vez una de las dos puertas metálicas que bloquean el acceso al jefe y una puerta de atajo, por la que el jugador puede bajar una escalera y volver directamente a la sala amplia sin repetir el recorrido.

La salida derecha conduce a la zona de rango. Tras cambiar de sala, el jugador encuentra al primer esqueleto mago; pasado este, una escalera sube a un segundo piso de la misma zona, donde esperan tres esqueletos mago dispuestos sobre plataformas, cada una con una rampa en su parte posterior que permite subir y atacarlos cuerpo a cuerpo, sin perjuicio de poder seguir atacándolos a distancia con la bola de fuego. Al final de esta sala, otra palanca abre la segunda puerta metálica de acceso al jefe junto con su propia puerta de atajo, que de igual forma permite bajar y regresar a la sala amplia.

Una vez abiertas ambas puertas metálicas -una desde cada zona- el jugador puede finalmente acceder a la sala del jefe.

Todo lo que el jugador hace en este nivel -cómo enfrenta a cada arquetipo, cómo esquiva, etc.- queda registrado y se utiliza más adelante para ajustar el comportamiento del jefe; el mecanismo concreto de ese ajuste se explica en el capítulo 4. Por ahora basta con entender este nivel como el último punto de referencia sobre el jugador antes de que comience el combate final.

3.3.3. Arena del jefe

A diferencia de los dos niveles anteriores, la arena del jefe se diseñó como un espacio simple: un pasillo corto conecta la sala del nivel previo con una sala de gran tamaño donde ocurre el combate. Al entrar a esta sala, una puerta metálica se cierra detrás del jugador, cortando la posibilidad de retirada, y se activa el jefe. No hay aquí una progresión de salas ni elementos adicionales que aprender -ese rol ya lo cumplieron los niveles anteriores-, sino un único espacio amplio que le da al jefe, versátil y con ataques de distinto alcance, el lugar necesario para desplegar todo su repertorio.

El detalle de la lógica propia de cada nivel -puertas, palancas y demás elementos reutilizables- se documenta en el capítulo 5.

Capítulo 4

Sistema adaptativo basado en el comportamiento del jugador

Capítulo 5

Implementación del videojuego

Para el proyecto se decidió utilizar Unreal Engine 5.6. El juego se construyó tomando como base la plantilla *Third Person* incluida en Unreal Engine. Esta corresponde a un ejemplo diseñado para demostrar las funcionalidades fundamentales del sistema de personajes en tres dimensiones, incorporando un escenario simple, una cámara orbital y un personaje capaz de desplazarse, correr y saltar. Si bien a primera vista los elementos presentados parecen utilizables directamente para la implementación del proyecto, la plantilla posee diversos aspectos que difieren del objetivo buscado y que requieren modificaciones específicas.

La plantilla *Third Person* proporciona una estructura de proyecto preconfigurada que incluye varios componentes clave. El elemento principal es el `BP_ThirdPersonCharacter`, un Blueprint que representa al personaje jugable y contiene toda la lógica de movimiento, entrada de usuario y comportamiento básico.

A lo largo de esta sección se hará referencia frecuente al concepto de Blueprint. Los Blueprints son el sistema de *scripting* visual de Unreal Engine que permite crear lógica de juego mediante nodos interconectados, sin necesidad de escribir código tradicional. Esto facilita la iteración rápida y permite a diseñadores y artistas contribuir directamente a la programación del juego.

Este Blueprint incluye componentes como la cápsula de colisión, la malla del personaje (*Skeletal Mesh*), el componente de movimiento (*Character Movement Component*) y la cámara con su brazo de resorte (*Spring Arm*). Adicionalmente, la plantilla incluye el *Game Mode*, que define las reglas básicas del juego, y diversos assets como animaciones base, materiales y el escenario de demostración.

Un ejemplo de los ajustes necesarios es la estructura del personaje y sus sistemas de locomoción, los cuales, si bien sirven como referencia inicial, deben ser adaptados o reemplazados según las necesidades mecánicas del juego. Del mismo modo, el comportamiento de la cámara, las animaciones base y la lógica de entrada proporcionadas por defecto funcionan como punto de partida, pero no necesariamente representan la experiencia final deseada.

Por ejemplo, se tuvieron que cambiar las animaciones de movimiento del personaje por algunas que soportaran *Strafe Movement*. El *Strafe Movement* (o simplemente *strafing*) consiste en moverse lateralmente sin perder la orientación hacia un objetivo específico. En lugar de girar todo el cuerpo para caminar hacia la izquierda o la derecha, el personaje desliza su movimiento lateral mientras mantiene la mirada fija hacia adelante o hacia un blanco. Este tipo de movimiento es común en juegos de acción y combate, ya que permite al jugador mantener contacto visual con enemigos mientras se desliza en cualquier dirección.

Se importaron animaciones que soportan el *Strafe Movement*, con las opciones de *Root Motion* y *Enable Root Motion* habilitadas. El *Root Motion* permite que el movimiento del personaje sea controlado directamente por las animaciones en lugar de ser aplicado mediante código, lo que resulta en desplazamientos más naturales y sincronizados con las animaciones reproducidas.

Lo siguiente fue crear un *Blend Space* asociado al esqueleto del jugador, para poder transicionar animaciones dependiendo de la velocidad y dirección del jugador. Un *Blend Space* es un asset de Unreal Engine que permite interpolar suavemente entre múltiples animaciones basándose en valores de entrada. En este caso, se configuró con dos ejes: uno horizontal que representa la dirección lateral del movimiento y uno vertical que representa la velocidad hacia adelante o atrás.

En el *Animation Blueprint* del personaje (*ABP_Manny*), específicamente en el *Animation Graph* dentro de la máquina de estados de *Locomotion*, se reemplazó la animación de *Idle* que venía en el template por una propia, perteneciente al conjunto de animaciones del *Strafe Movement*.

Luego, se reemplazó en el estado de *Walking/Run* el *Blend Space* que venía en el template por el *Blend Space* creado previamente, permitiendo así transiciones fluidas entre las diferentes direcciones de movimiento mientras se mantiene la orientación del personaje. También se desactivó el parámetro «Orient Rotation to Movement» y se activó el parámetro «Use Controller Desired Rotation» para que el jugador mire siempre hacia la dirección que indica la cámara, independientemente de la dirección en la que se mueva.

Cabe recalcar que la mayoría del desarrollo de las implementaciones está realizado en Blueprint. Sin embargo, en el futuro se espera incorporar componentes heredados de C++.

Para desarrollar las capacidades de combate del jugador, se decidió crear un componente `BPC_Combat` que hereda de *Actor Component* y se ancla al Blueprint del jugador, de tal forma que esté centralizada la lógica de combate. De igual forma, para las estadísticas del jugador se creó y ancló un componente `BPC_Stats`.

5.1. Estadísticas del jugador

Como se mencionó anteriormente, las estadísticas del jugador se implementaron en el componente `BPC_Stats` anclado al jugador. Este componente posee múltiples variables, entre las cuales están:

1. `health`: Tipo *Float*. Representa la vida actual del jugador.
2. `Progress Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra de progreso que muestra la vida.
3. `maxHealth`: Tipo *Float*. Representa el valor máximo de vida del jugador.
4. `stamina`: Tipo *Float*. Representa la stamina actual del jugador.
5. `maxStamina`: Tipo *Float*. Representa la stamina máxima del jugador.
6. `Stam Bar Ref`: Tipo *WB Player Stamina*. Es la referencia al widget que muestra la barra de stamina.
7. `canRecoverStamina`: Tipo *Boolean*. Indica si el jugador puede recuperar stamina.
8. `maxMana`: Tipo *Float*. Representa el valor máximo de maná del jugador.
9. `mana`: Tipo *Float*. Representa el maná actual del jugador.
10. `Mana Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra que muestra el maná.

La funcionalidad principal de este componente es una función llamada `IncreaseVal`, la cual recibe como parámetros un enum que indica la estadística que se quiere modificar, junto con el valor del cambio (positivo para incrementar, negativo para disminuir). Según el valor del enum, se actualiza la estadística correspondiente sumándole el valor recibido, acotando siempre el resultado entre cero y su valor máximo (`maxHealth`, `maxMana` o `maxStamina`, según corresponda) mediante la función `FClamp`, de modo que ninguna estadística pueda exceder su máximo ni quedar en negativo.

Si la estadística modificada es la vida (`health`), tras actualizarla se realizan dos acciones en paralelo: por un lado, se comprueba si la vida resultante es menor o igual a cero, y de ser así se invoca el evento `Die`, que da inicio a la secuencia de muerte del jugador; por otro, se calcula el porcentaje de vida actual (`health/maxHealth`) y, desde la referencia a la barra (`Progress Bar Ref`), se llama al método `Set Percent`, que actualiza visualmente la barra con dicho porcentaje. En caso de recibir el enum perteneciente a la estadística de maná, el proceso de actualización visual es análogo.

Para la estadística de stamina también es similar, con la diferencia de que el widget de la stamina no se muestra siempre, solo cuando se está gastando o recuperando stamina,

similar a como lo implementan juegos como *The Legend of Zelda: Breath of the Wild*⁹.

Así, al recibirse el enum de stamina, se debe revisar si existe un widget creado en pantalla. Si se posee, simplemente se le indica al widget que cambie su porcentaje y al final se revisa si el jugador posee toda la stamina. En caso contrario, significa que el widget no ha sido creado, por lo que se crea y se añade a la pantalla.

Finalmente, se revisa si la stamina es mayor o igual al máximo (100%). En caso de ser verdadero, significa que el jugador ya posee toda la stamina, por lo que se procede a remover el widget de la pantalla.

Al iniciarse el componente este activa dos timers que se ejecutan cada 0.01 segundos y llaman a los eventos `RegainStamina` y `RegainMana`. Estos eventos pasan por una rama que verifica si corresponde regenerar la estadística respectiva; si la condición es verdadera, se usa el nodo `IncreaseVal` para aumentar la stamina o el maná en pequeñas cantidades, creando así un sistema de regeneración continua controlada por condiciones. Es importante mencionar que este enfoque de regeneración mediante timers de alta frecuencia podría optimizarse en el futuro.

5.2. Recepción de daño y muerte

El procesamiento del daño y la curación que recibe el jugador se centraliza en el evento `ReceiveAnyDamage` de `BP_ThirdPersonCharacter`, el cual es invocado automáticamente por el motor cada vez que se llama a la función `ApplyDamage` sobre el jugador, sin importar el origen del daño. Este evento recibe el valor a aplicar y, según su signo, distingue entre dos casos.

Si el valor es negativo, se interpreta como curación: se invoca directamente `IncreaseVal` sobre `BPC_Stats` con el valor invertido, restaurando la vida del jugador según lo descrito en la sección anterior.

Si el valor es positivo, se interpreta como daño recibido. Antes de procesarlo, se comprueba si el jugador se encuentra esquivando (`isDodging`); de ser así, la función retorna de inmediato sin aplicar daño, lo que implementa la invulnerabilidad del jugador durante toda la esquivada descrita en el capítulo 3.

En caso contrario, se ejecuta una secuencia más elaborada. En primer lugar, se intenta identificar si el causante del daño es el jefe (`BP_Slime`); de ser así, se consulta su *Behaviour Tree* para obtener cuál fue su último ataque (`LastAttack`) y se registra en el `PlayerMetricsComponent` mediante `RegisterBossAttackHit`, asociando así cada golpe

⁹The Legend of Zelda: Breath of the Wild es un videojuego de acción y aventuras en mundo abierto desarrollado y publicado por Nintendo en 2017. El juego introduce mecánicas innovadoras de supervivencia y exploración, incluyendo un sistema de stamina que se muestra dinámicamente solo cuando se está utilizando o recuperando. Disponible en sitio web.

recibido con el tipo de ataque que lo causó. A continuación, se distingue si el jugador se encontraba esquivando en el momento del impacto (`isDodging`): si es así, se registra como un dodge fallido mediante `RegisterDodgeResult`; en caso contrario, se registra el daño normalmente mediante `RegisterDamageTaken`, y se reproduce el sonido de dolor correspondiente. Si además el jugador se encontraba dentro de una zona específica del combate (`isInTankZone`), se efectúa un registro adicional (`RegisterDamageTakenInTankZone`).

Finalmente, se aplica el daño a la vida del jugador mediante `IncreaseVal` y, si el daño es mayor a cero, se reproduce la animación de reacción a daño (`HitReact_Montage`), que interrumpe temporalmente el movimiento del jugador. Esta interrupción respeta las acciones que ya estén en curso: si el jugador se encuentra en medio de la animación de beber poción, de lanzar un hechizo, o del combo de ataque con espada, el `HitReact_Montage` no detiene el movimiento, evitando que la reacción a daño corte abruptamente otra acción ya iniciada.

Cuando la vida del jugador llega a cero, `IncreaseVal` invoca el evento `Die`, que gestiona la secuencia de derrota. Este evento desactiva el componente de combate, reproduce una animación de derrota (`MO_DefeateDaiko`) y, una vez finalizada, pausa el juego, muestra el widget de «Try Again» y habilita el cursor del mouse. Si la derrota ocurrió en el nivel del jefe, adicionalmente se vuelca a un archivo el registro de resultados del combate mediante `DumpCombatResultToFile`, dato que forma parte del sistema de recolección de métricas del estudio de usuario.

5.3. Componente de combate

Como se mencionó anteriormente, se creó un *Actor Component* llamado `BPC_Combat` que se ancla al jugador y centraliza toda la lógica relacionada con el combate.

Este componente posee múltiples variables, entre las cuales están:

1. `TargetLock`: Tipo *Actor*. Es la referencia al objetivo que se tiene fijado por la cámara.
2. `Player BP`: Tipo *Object Reference a BP Third Person Character*. Referencia al jugador.
3. `TargetLockWidget`: Tipo *Object Reference a BP Target Lock Widget*. Es la referencia al widget del símbolo que se crea cuando se fija un objetivo.
4. `isAttacking`: Tipo *Boolean*. Es una bandera que se levanta cuando el jugador está atacando.
5. `isCombo`: Tipo *Boolean*. Bandera que se levanta cuando el jugador tiene la intención de continuar el ataque.
6. `isDodging`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está realizando una esquivas.
7. `StamWidget`: *Object Reference a WB Player Stamina*. Es la referencia al widget de la stamina que se muestra cuando se gasta o recupera stamina.
8. `StaminaRollUse`: Tipo *Float*. Define la cantidad de stamina que consume la acción de rodar.

9. `isRunning`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está corriendo.
10. `StaminaRunUse`: Tipo *Float*. Define la cantidad de stamina que gasta el jugador al correr.
11. `runSpeed`: Tipo *Float*. Define la velocidad del jugador al correr.
12. `CurrentSpellIndex`: Tipo *Float*. Índice que indica el hechizo que tiene equipado el jugador actualmente.
13. `SpellImageObject`: Tipo *Image*. Referencia a la imagen que indica el hechizo equipado actualmente en la interfaz.
14. `magicManaUse`: Tipo *Float*. Define la cantidad de maná que consume el proyectil mágico del jugador.
15. `healingManaUse`: Tipo *Float*. Define la cantidad de maná que consume el hechizo de curación.
16. `SwordDamage`: Tipo *Float*. Define el daño base que inflige la espada del jugador.
17. `currentLever`: Tipo *Object Reference* a *BP Lever*. Referencia a la palanca con la que el jugador está interactuando actualmente.
18. `isEnemyDashing`: Tipo *Boolean*. Bandera que indica si el enemigo está realizando una embestida o desplazamiento rápido.
19. `isInCorrectDodgeWindow`: Tipo *Boolean*. Bandera que indica si el jugador se encuentra dentro de la ventana temporal adecuada para ejecutar una esquivada efectiva.
20. `isRollingSideWays`: Tipo *Boolean*. Bandera que indica si la esquivada actual se está realizando lateralmente.
21. `MovInput`: Tipo *Vector2D*. Almacena la dirección de movimiento introducida por el jugador.
22. `FireBallImage`: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar el hechizo de bola de fuego.
23. `PotionImage`: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar las pociones del jugador.
24. `PotiCount`: Tipo *Integer*. Indica la cantidad actual de pociones disponibles.
25. `Potion Count Text Var`: Tipo *Text*. Referencia al elemento de texto de la interfaz que muestra la cantidad de pociones disponibles.
26. `healingPotion`: Tipo *Static Mesh*. Referencia al modelo 3D de la poción de curación utilizada por el jugador.
27. `tmpTarget`: Tipo *Vector*. Almacena una ubicación temporal utilizada por los enemigos como objetivo durante la ejecución de sus comportamientos.

5.4. Sistema de input

La entrada del jugador se gestiona mediante el sistema *Enhanced Input* de Unreal Engine. Cada acción del jugador (atacar, esquivar, correr, etc.) se representa como un *Input Action* independiente, desacoplado de la tecla o botón físico que lo activa. Esta asociación entre la entrada física y el *Input Action* correspondiente se define en un *Input Mapping Context*, lo que facilita el remapeo de controles sin modificar la lógica interna de cada Blueprint. Actualmente el juego solo cuenta con soporte para teclado y mouse.

Los **Input Actions** utilizados por el componente de combate (**BPC_Combat**) y sus controles asociados son los siguientes:

Input Action	Control
IA_Attack	Click izquierdo
IA_Dash-Roll	Barra espaciadora
IA_Run	Shift
IA_Magic	Q
IA_ScrollMagic	Rueda del mouse
IA_Lock	Click rueda del mouse
IA_MyInteract	E
IA_EscapeMenu	Escape

En las secciones siguientes, cada acción del jugador se describe haciendo referencia explícita al **Input Action** correspondiente (por ejemplo, **IA_Attack** o **IA_Dash-Roll**), ya activado mediante el *Input Mapping Context* descrito anteriormente.

5.5. Ataque melee del jugador

El ataque del jugador se implementó en el componente de combate mencionado previamente. Al recibirse el **Input Action** **IA_Attack**, antes de ejecutar la acción se comprueba que el jugador no esté ya atacando ni esquivando (variables **isAttacking** e **isDodging**) y, además, que no se esté reproduciendo el *Animation Montage* de reacción a daño (**HitReact_Montage**), de modo que el jugador no pueda atacar mientras recibe un golpe. Si la condición se cumple, se setea **isAttacking** como **true**, se registra el ataque cuerpo a cuerpo en el *Game Instance* (para fines de telemetría) y se ejecuta el *Animation Montage* asociado al ataque, que contiene una secuencia de tres golpes consecutivos.

El encadenamiento de golpes (combo) se controla mediante la variable **isCombo**, que representa la intención del jugador de continuar la secuencia. Esta variable se activa de forma diferida: si el jugador vuelve a recibir el **Input Action** **IA_Attack** mientras un ataque ya está en curso (es decir, cuando la condición inicial es **false** porque **isAttacking** es **true**), en lugar de iniciar un ataque nuevo se setea **isCombo** como **true**. De esta forma el sistema «almacena» la intención de combo hasta el siguiente punto de decisión.

Dicho punto de decisión se evalúa mediante un *Animation Notify* ubicado en el montage, al final del primer y segundo golpe. Cada vez que el notify se activa, se comprueba si el jugador sigue atacando y si tiene la intención de continuar (variables **isAttacking** e **isCombo**). Si ambas son verdaderas, se permite que la secuencia continúe y se resetea **isCombo** a **false** (a la espera de un nuevo input). En caso contrario, se detiene el montage prematuramente mediante **Montage_Stop** y se limpian las variables **isAttacking** e **isCombo**. Cuando el montage se completa o es interrumpido, se restablecen también **isAttacking**

e `isCombo` a `false` y se restaura el estado de movimiento del personaje; en el caso de interrupción, se distingue además si la causa fue la reproducción del `HitReact_Montage`.

Con respecto a la detección de impacto del ataque, la implementación se basa en un sistema de *trace* por temporizador. El *montage* tiene asociado un *Blueprint* que hereda de *Animation Notify State*, el cual delimita la ventana en la que el arma debe poder golpear mediante un momento de inicio y un momento de fin dentro del *montage*. Al recibirse la señal de inicio, se invoca el evento **Begin Damage Trace**, que inicia un *timer* en bucle que, cada 0.1 segundos, ejecuta un *Sphere Trace* entre dos componentes de escena anclados al arma (**Start Sword Trace Pos** y **End Sword Trace Pos**), que marcan la base y la punta de la hoja. Si el *trace* detecta una colisión válida, se llama a la función **ApplyDamage** sobre el actor impactado, utilizando el valor de la variable **SwordDamage** y al jugador como causante del daño. Al recibirse la señal de fin, se invoca el evento **End Damage Trace**, que invalida el *timer*, desactivando la detección hasta el siguiente golpe.

5.6. Projectiles

Tanto el jugador como el jefe pueden lanzar proyectiles. Es por esto que se decidió crear un *Blueprint Class* que represente la base de un proyectil (**BP_BaseProjectile**), para que después tanto el proyectil del jugador como el del jefe hereden desde esta clase base y ajusten variables específicas de cada uno. El comportamiento por defecto del proyectil es avanzar con velocidad constante en línea recta hasta chocar con algo, momento en el cual reproduce el efecto visual (VFX) y el sonido de impacto correspondientes, aplica daño al objetivo impactado y se destruye.

El proyectil base posee múltiples componentes, entre los cuales están:

1. **BoxCollision**: Componente de colisión (*Box Collision*). Define el volumen de colisión principal del proyectil.
2. **Arrow**: Componente *Arrow*. Sirve como referencia visual y direccional para indicar hacia dónde se moverá u orientará el proyectil.
3. **StaticMesh**: Componente *Static Mesh*. Representa la malla estática visible del proyectil, en caso de necesitarse.
4. **ProjectileMovement**: Componente *Projectile Movement*. Controla el movimiento del proyectil, incluyendo velocidad, gravedad y comportamiento de persecución (*homing*).

Las variables de las que se compone son:

1. **Speed**: Tipo *Float*. Define la velocidad a la que se mueve el proyectil.
2. **Gravity**: Tipo *Float*. Controla cuánta influencia tiene la gravedad sobre el proyectil.
3. **Homing**: Tipo *Boolean*. Indica si el proyectil tiene la capacidad de perseguir automáticamente a un objetivo.
4. **Base Damage**: Tipo *Float*. Define el daño base que causa el proyectil al impactar.

5. **Impact Effect:** Tipo *Particle System*. Es la referencia al sistema de partículas que se reproduce cuando el proyectil impacta.
6. **Sound Impact:** Tipo *Sound Base*. Es la referencia al sonido que se reproduce cuando el proyectil impacta.

El comportamiento del proyectil base se define mediante los siguientes eventos. Al iniciarse (**BeginPlay**), el proyectil configura su componente de colisión para ignorar al actor que lo originó (**GetOwner**), evitando así que colisione consigo mismo o con quien lo disparó. Además, si la variable **Homing** está activada, se habilita el modo de persecución del componente *Projectile Movement* (**bIsHomingProjectile**) y se establece el componente objetivo de la persecución.

La detección de impacto se gestiona mediante un evento ligado al solapamiento del componente de colisión. Cuando el proyectil se solapa con otro actor, se comprueba que dicho actor no sea su propio dueño (**GetOwner**); de no serlo, se invoca la función **SpawnImpactEffect**, que reproduce el sistema de partículas **Impact Effect** en el punto de colisión junto con el sonido **Sound Impact**. A continuación se aplica el daño definido en **Base Damage** sobre el actor impactado —indicando al dueño del proyectil como causante del daño— y finalmente el proyectil se destruye.

5.6.1. Proyectil del jugador

Uno de los hechizos que puede utilizar el jugador es el de disparar un proyectil. El Blueprint que representa al proyectil del jugador (**BP_PlayerFireBall**) hereda de **BP_BaseProjectile** y configura las siguientes variables: **Speed** con un valor alto (para que sea más rápido que el proyectil del jefe), **Gravity** en 0 (sin influencia gravitacional), **Homing** como **false** (no persigue objetivos), y el **Impact Effect** con un VFX de explosión de fuego. En cuanto a los componentes visuales, el **StaticMesh** permanece vacío y en su lugar se utiliza un componente *Particle System* que representa el efecto visual del proyectil en movimiento.

5.6.2. Proyectil del jefe

El jefe puede lanzar proyectiles mediante ciertos ataques. El Blueprint que los representa (**BP_BossSlimeBall**) hereda de **BP_BaseProjectile** y configura sus variables de la siguiente manera: **Speed** con un valor moderado (menor que el del jugador), **Gravity** en 0, el **Impact Effect** con un VFX de explosión de slime, y **Homing** que varía según el tipo de ataque. El jefe dispone de dos variantes de ataque con proyectiles: uno donde el proyectil avanza en línea recta a velocidad constante (**Homing** en **false**), y otro donde persigue activamente al jugador hasta alcanzarlo o explotar tras un tiempo determinado (**Homing** en **true**). Al igual que el proyectil del jugador, el **StaticMesh** permanece vacío y se emplea un componente *Particle System* para el efecto visual del proyectil en movimiento.

Esta subclase sobrescribe (*override*) el evento **BeginPlay** para extender el comportamiento heredado: al instanciarse, primero ejecuta el **BeginPlay** de la clase base (configurando la colisión y el *homing* según corresponda) y, a continuación, inicia un temporizador

de 5 segundos. Transcurrido ese tiempo, si el proyectil aún no ha impactado contra nada, spawna un sistema de partículas Niagara (`NS_Projectile_03_Hit`) en su ubicación actual y se autodestruye, garantizando así que los proyectiles que no alcancen al jugador no permanezcan indefinidamente en la escena.

5.7. Hechizos y consumibles del jugador

El jugador dispone de dos opciones que puede alternar y utilizar mediante una misma acción: un hechizo de proyectil de fuego y una poción de vida. Esta mecánica se implementó con el propósito de otorgar al jugador versatilidad en combate —pudiendo elegir entre una opción ofensiva y una defensiva— y de proporcionar al jefe la oportunidad de demostrar capacidad de aprendizaje y adaptación a las diferentes estrategias del jugador.

El jugador puede intercambiar entre el hechizo de fuego y la poción utilizando la rueda del mouse, y la opción actualmente equipada se muestra representada con un símbolo en la esquina inferior derecha de la pantalla. La selección se representa mediante la variable `CurrentSpellIndex`, que cambia al mover la rueda del mouse (`IA_ScrollMagic`); además de actualizar el valor, se modifica el ícono y la opacidad de la imagen del HUD para reflejar la opción activa.

Cuando el jugador ejecuta la acción de lanzar (`IA_Magic`), el sistema evalúa la variable `CurrentSpellIndex` para determinar qué opción está equipada. Si corresponde al hechizo de fuego, se comprueba primero que se disponga del maná necesario (`magicManaUse` frente a la variable `mana`) antes de invocar el evento del proyectil; si corresponde a la poción, se invoca directamente el evento de curación, cuya disponibilidad depende de la cantidad de pociones que posea el jugador.

5.7.1. Hechizo de proyectil de fuego

El evento del hechizo de proyectil (`ProjMagicEvent`) comienza comprobando que el jugador no esté atacando ni esquivando, y que no se esté reproduciendo la animación de reacción a daño. Si la condición se cumple, registra el ataque a distancia en el *Game Instance* con fines de telemetría, marca al jugador como atacando y reproduce la animación de lanzamiento, que corresponde a un *Animation Montage*. Durante la reproducción se desactiva temporalmente el movimiento del jugador, ya que mientras lanza el proyectil no debe poder desplazarse; el movimiento se restablece cuando la animación finaliza o es interrumpida.

El proyectil en sí se genera mediante un *Animation Notify* anclado al montage: en el instante en que la animación lo indica, se instancia el proyectil del jugador (`BP_PlayerFireBall`) con el jugador asignado como su dueño (*Owner*), de modo que el proyectil no colisione con quien lo dispara. En ese mismo momento se reproduce el sonido de lanzamiento y se realiza el consumo de maná, llamando a la función `IncreaseVal` del componente `BPC_Stats`, pasando como parámetro el valor de la variable `magicManaUse` con signo negativo para representar la disminución.

5.7.2. Poción de vida

La poción de vida constituye la opción defensiva del jugador. No restaura la vida de forma instantánea, sino que requiere que el jugador beba una poción de una cantidad limitada que posee, representada por la variable `PotiCount` y mostrada en el HUD. El evento correspondiente (`HealingMagicEvent`) comienza comprobando que el jugador no esté atacando ni esquivando, que no se esté reproduciendo la animación de reacción a daño y, fundamentalmente, que disponga de al menos una poción (`PotiCount` mayor que cero).

Si la condición se cumple, se registra el momento de curación en el *Game Instance* (junto con la vida actual y máxima del jugador), se decrementa el contador de pociones y se actualiza el HUD, y se desactiva el movimiento mientras se reproduce la animación de beber, que corresponde a un *Animation Montage*. Para reforzar visualmente la acción, durante la animación se genera y se ancla la malla de la poción a la mano del jugador, la cual se destruye una vez finalizada. La restauración de vida se realiza mediante un *Animation Notify* anclado al montage: en el momento en que la animación muestra al jugador bebiendo, se llama a la función `IncreaseVal` para restaurar la vida del jugador. Al completarse o interrumpirse la animación, se restablece el movimiento del jugador.

5.8. Esquiva

La esquiva del jugador está implementada en el componente `BPC_Combat`. Se creó una función auxiliar (`GetRollMontage`) para determinar qué animación de roll ejecutar, ya que se puede realizar un roll hacia adelante, hacia atrás, o hacia los lados cuando se tiene un objetivo fijado mediante el sistema de *Target Lock*.

La función `GetRollMontage` determina cuál de las cuatro animaciones de roll ejecutar a partir de la dirección de movimiento del jugador. Para ello construye el vector de intención de movimiento en espacio mundo, combinando la entrada de movimiento (`MovInput`, actualizada continuamente mediante el `Input Action IA_Move`) con la rotación de la cámara (la rotación del controlador), y lo normaliza. Luego proyecta ese vector sobre el *Forward Vector* y el *Right Vector* del personaje mediante productos punto, y compara las magnitudes de ambas proyecciones: si la componente hacia adelante/atrás es mayor o igual que la lateral, el roll se considera frontal; en caso contrario, lateral.

Una vez determinado el eje dominante, el signo del producto punto correspondiente define el sentido exacto y, con ello, el *Animation Montage* que se retorna: hacia adelante (`Dodge_F_MontageDAIKO`) o hacia atrás (`Dodge_B_MontageDAIKO`) en el caso frontal, y hacia la derecha (`Dodge_R_MontageDAIKO`) o hacia la izquierda (`Dodge_L_MontageDAIKO`) en el caso lateral. Cuando el roll es lateral, además se marca la variable `isRollingSideWays` como `true`.

Adicionalmente, esta función realiza un registro de telemetría específico para las esquivas realizadas frente a una embestida del jefe: si en el momento del roll el jefe se encuentra embistiendo (`isEnemyDashing`) y el jugador está dentro de una zona determinada (`isInTankZone`), se llama a la función `RegisterDashDodge` del *Game Instance*, pasándole

el parámetro `bWasLateral` para indicar si la esquivas fue lateral (`true`, en los rolls hacia los lados) o frontal (`false`, en los rolls hacia adelante o atrás). Esto permite recopilar métricas diferenciadas sobre cómo responde el jugador ante el ataque de embestida según la dirección de su evasión.

En cuanto a la lógica del roll: cuando se recibe el `Input Action IA_Dash-Roll`, primero se comprueba que el jugador no esté atacando (variable `isAttacking`), que disponga de la stamina suficiente para realizar la esquivas (comparando la `stamina` actual contra `StaminaRollUse`) y que no se esté reproduciendo la animación de reacción a daño (`HitReact_Montage`). Si la condición se cumple, se registra la esquivas en el *Game Instance* mediante la función `RegisterTotalDodge` y se procede a ejecutar la acción: se llama a la función `IncreaseVal` para restar la stamina consumida, se marca `isDodging` como `true`, se reproduce el sonido de esquivas y se ejecuta la animación de roll correspondiente (obtenida mediante `GetRollMontage`). Inmediatamente se setea la variable `canRecoverStamina` como `false` para evitar que se regenere stamina durante la esquivas.

La limpieza de los estados se realiza al término de la animación de roll. Tanto si el montage se completa exitosamente como si es interrumpido, se restablece `canRecoverStamina` a `true` (permitiendo nuevamente la regeneración de stamina) y se limpian las variables `isDodging` e `isRollingSideWays`, devolviendo al jugador a un estado neutro desde el cual puede volver a actuar.

Adicionalmente, el sistema distingue si la esquivas se realizó dentro de una ventana de tiempo considerada como una evasión precisa o «perfecta», mediante la variable `isInCorrectDodgeWindow`. Cuando este es el caso, se efectúa un registro extra en el *Game Instance* mediante la función `RegisterDodgeFromDelayAttack`, lo que permite recopilar métricas sobre las esquivas realizadas con buen *timing* frente a los ataques enemigos.

Cabe mencionar que todo esto se ejecuta dentro de un nodo *Do Once*, el cual, como bien indica su nombre, solo se ejecuta una vez hasta que se resetee. El reseteo se realiza después de 0.45 segundos mediante un nodo *Delay*. Esto permite que el jugador pueda hacer un roll cada 0.45 segundos, pudiendo así encadenar rolls consecutivos para evadir múltiples ataques.

5.9. Fijado de objetivo (Target Lock)

El sistema de fijado de objetivo (*Target Lock*) permite al jugador centrar la cámara sobre un enemigo y mantenerla orientada hacia él durante el combate, facilitando el seguimiento del objetivo mientras se ataca o esquivas. Está implementado en el componente `BPC_Combat` y se reparte entre dos partes: la activación/desactivación del fijado, gestionada por el `Input Action IA_Lock`, y el mantenimiento de la orientación de la cámara, ejecutado en cada fotograma dentro del evento `Tick`.

Al recibirse el `Input Action IA_Lock`, el sistema actúa según si ya existe un objetivo fijado, almacenado en la variable `Target Lock`. Si ya hay un objetivo, se interpreta como una orden de soltarlo: se limpia la referencia, se restauran los parámetros de rotación del personaje a su comportamiento normal (orientación según el movimiento) y se destruye el widget visual del fijado. Si no hay objetivo, se procede a buscar uno: se realiza un *Sphere Trace* hacia adelante desde el personaje (con un radio y un alcance determinados); si dicho trazado detecta un objetivo válido, se almacena en `Target Lock`, se ajustan los parámetros de rotación para que el personaje se oriente hacia el objetivo, y se genera un widget (`BP_TargetLockWidget`) que se ancla al enemigo como indicador visual del fijado.

Una vez fijado el objetivo, su seguimiento se realiza en el evento `Tick`. Mientras exista un objetivo, cada fotograma se calcula la rotación necesaria para que la cámara apunte hacia él y se aplica a la rotación del controlador del jugador. Para obtener el punto exacto al que mirar, el sistema comprueba si el objetivo implementa la interfaz `BPI_Lockable`: si la implementa, solicita a través de ella un punto de fijado personalizado, lo que permite que ciertos enemigos —como aquellos cuya geometría se deforma o cuyo centro visual no coincide con su origen— definan explícitamente dónde debe apuntar la cámara; si no la implementa, se utiliza directamente la ubicación del objetivo. Además, mientras el fijado está activo, se reposiciona en pantalla el widget de la barra de stamina.

Actualmente, la única clase que implementa la interfaz `BPI_Lockable` es el jefe (`BP_Slime`). Esto se debe a que su animación de ataque de charco desplaza visualmente al personaje hacia abajo —simulando que se hunde o agacha en el charco— sin que dicho desplazamiento se traduzca en un movimiento real del actor en el mundo. Si la cámara utilizara directamente la ubicación del actor (su origen), el punto de fijado quedaría desalineado respecto a la posición visual del jefe durante esta animación. Al implementar `BPI_Lockable`, el jefe puede reportar un punto de fijado ajustado a su posición visual real en cada momento, manteniendo la cámara correctamente orientada incluso cuando la animación no coincide con la posición lógica del actor.

5.10. Correr

La carrera permite al jugador desplazarse a mayor velocidad a cambio de consumir stamina, y está implementada en el componente `BPC_Combat` mediante el `Input Action IA_Run`. A diferencia de otras acciones, esta responde a las distintas fases del input —el momento en que se presiona, mientras se mantiene presionado, y cuando se suelta—, lo que permite controlar tanto la activación como el gasto continuo de stamina y el regreso al estado normal.

Como condición común a todas las fases, la carrera solo opera si no se está reproduciendo la animación de reacción a daño (`HitReact_Montage`), de modo que recibir un golpe interrumpe la posibilidad de correr.

Al presionar el input, si el jugador dispone de stamina suficiente, se aumenta su velocidad de movimiento (`MaxWalkSpeed`) al valor definido por la variable `runSpeed` y se bloquea la regeneración de stamina (`canRecoverStamina` en `false`), ya que no debe recuperarse stamina mientras se corre. Mientras el input se mantiene presionado, y siempre que aún quede stamina, se consume de forma continua llamando a la función `IncreaseVal` del componente `BPC_Stats` con el valor de `StaminaRunUse` en negativo, lo que reduce progresivamente la stamina disponible durante la carrera. Finalmente, al soltar el input, se restaura la velocidad de movimiento a su valor normal y se vuelve a habilitar la regeneración de stamina, devolviendo al jugador a su estado de desplazamiento habitual.

5.11. Interactuar

La interacción del jugador con elementos del entorno está implementada en el componente `BPC_Combat` mediante el `Input Action IA_MyInteract`. Actualmente esta acción está orientada a la interacción con palancas: al recibirse el input, primero se comprueba que exista una palanca con la que el jugador pueda interactuar en ese momento, mediante la referencia `currentLever`. Si existe dicha referencia, se procede a verificar si ya fue utilizada (`bAlreadyUsed`); de no haberlo sido, se invoca la función `PlayLever` sobre el Blueprint de la palanca (`BP_Lever`), que se encarga de reproducir su animación y de ejecutar el efecto asociado a su activación.

5.12. Menú de pausa

El acceso al menú de pausa está implementado en el componente `BPC_Combat` mediante el `Input Action IA_EscapeMenu`. Al recibirse el input, se comprueba si el juego ya se encuentra pausado; si no lo está, se procede a abrirlo: se instancia el widget del menú de pausa (`WB_PauseMenu`) y se añade al *viewport*, se cambia el modo de entrada a uno orientado exclusivamente a la interfaz (de forma que el movimiento del mouse deje de controlar la cámara), se habilita la visibilidad del cursor, y finalmente se pausa el juego.

5.13. Widgets del jugador

La interfaz de usuario asociada al jugador se compone de varios *Widget Blueprints*, cada uno encargado de mostrar un aspecto específico del estado del jugador o de gestionar una pantalla concreta. La mayoría de estos widgets se actualizan desde la lógica de juego ya descrita —principalmente desde el componente `BPC_Stats`—, por lo que su Blueprint interno es mínimo y se limitan a su composición visual.

El widget principal es el HUD del jugador (`WB_PlayerHUD`), que actúa como contenedor de los elementos que se muestran de forma persistente durante el juego. Está compuesto por un *Canvas Panel* que agrupa la barra de vida (`WB_PlayerHealth`), la barra de maná (`WB_PlayerMana`) y el widget de equipamiento (`WB_Equips`). Este widget no posee lógica en su grafo de eventos, ya que su única función es disponer espacialmente en pantalla a los sub-widgets que contiene.

Los widgets de barra de vida (`WB_PlayerHealth`) y de maná (`WB_PlayerMana`) son estructuralmente simples: cada uno se compone de un *Canvas Panel* que contiene un borde decorativo y una barra de progreso (*Progress Bar*). No poseen lógica en su grafo de eventos, ya que su valor se actualiza directamente desde la función `IncreaseVal` de `BPC_Stats`, la cual modifica el porcentaje de la barra cada vez que cambia la estadística correspondiente, tal como se describió en la sección de estadísticas del jugador.

El widget de equipamiento (`WB_Equips`) muestra los consumibles y hechizos disponibles del jugador. Se compone de una caja horizontal con la imagen del hechizo de fuego (`Fireball_IMG`) y, superpuestos, la imagen de la poción (`Potion_IMG`) junto con un texto que indica la cantidad de pociones restantes (`PotionCountText`). Tampoco posee lógica en su grafo, pues tanto el ícono activo como el contador se actualizan desde `BPC_Combat`.

Finalmente, el widget de fijado de objetivo (`WB_TargetLock`) consiste únicamente en una imagen (un ícono) que se ancla sobre el enemigo fijado, según lo descrito en la sección de *Target Lock*.

5.13.1. Barra de stamina (`WB_PlayerStamina`)

A diferencia de las barras de vida y maná, la barra de stamina sí posee lógica propia en su Blueprint, ya que se representa mediante un material en lugar de una *Progress Bar* estándar. El widget se compone de una única imagen (`Image_176`) a la que se le aplica un material dinámico, lo que permite representar la barra de stamina con una forma circular.

La actualización del nivel de stamina se realiza mediante la función `SetPercent`. Esta función primero comprueba si la instancia dinámica del material (`RoundStaminaBarInst`) ya fue creada: si es válida, simplemente actualiza el parámetro escalar `Percent` del material con el nuevo valor. Si aún no existe, crea una instancia dinámica a partir del material base (`M_StaminaBar_Inst`), la asigna como *brush* de la imagen, y luego actualiza su parámetro `Percent`. De esta forma, la instancia del material se crea una sola vez (la primera vez que se actualiza la barra) y se reutiliza en las llamadas posteriores.

5.13.2. Widget del menú de pausa (`WB_PauseMenu`)

El widget del menú de pausa presenta dos botones dispuestos verticalmente: reanudar (`ResumeBtn`) y volver al menú principal (`MainMenuButton`). Su lógica responde a los eventos de pulsación de cada botón. Al presionar el botón de reanudar, se reanuda el juego (quitando la pausa), se restablece el modo de entrada a uno orientado exclusivamente al juego, se oculta el cursor del mouse, y se elimina el propio widget de la pantalla, devolviendo al jugador al combate. Al presionar el botón de menú principal, se reanuda el juego y se carga el nivel del menú principal (`Lvl_MainMenu`).

5.13.3. Pantalla de derrota (`WB_TryAgain`)

Esta pantalla se muestra cuando el jugador es derrotado e incluye un mensaje de «Game Over» junto con dos botones: reintentar (`Retry Button`) y volver al menú principal (`Main`

Menu Button). Al presionar reintentar, se reanuda el juego, se incrementa el contador de intentos (`AttemptNumber`) almacenado en el `SlimeGameInstance` —dato relevante para el seguimiento del estudio— y se recarga el nivel actual, permitiendo al jugador volver a enfrentar el combate. Al presionar el botón de menú principal, se reanuda el juego y se carga el nivel del menú principal.

5.14. Enemigos

5.14.1. Esqueleto normal

El enemigo más simple del juego es el esqueleto normal (`BP_Skeleton`), un enemigo de combate cuerpo a cuerpo cuya estructura de componentes y comportamiento sientan la base sobre la cual se construyen los demás enemigos. Se compone de una cápsula de colisión como componente raíz, sobre la cual se ancla un componente de barra de vida (*Widget Component*); una malla esquelética (`Mesh`) que incluye dos componentes de escena (`StartOfTrace` y `EndOfTrace`) ubicados en la espada, utilizados para la detección de impacto del ataque, y un componente `Arrow` como referencia direccional; un componente de movimiento de personaje; y un componente `PawnSensing`, encargado de la detección del jugador.

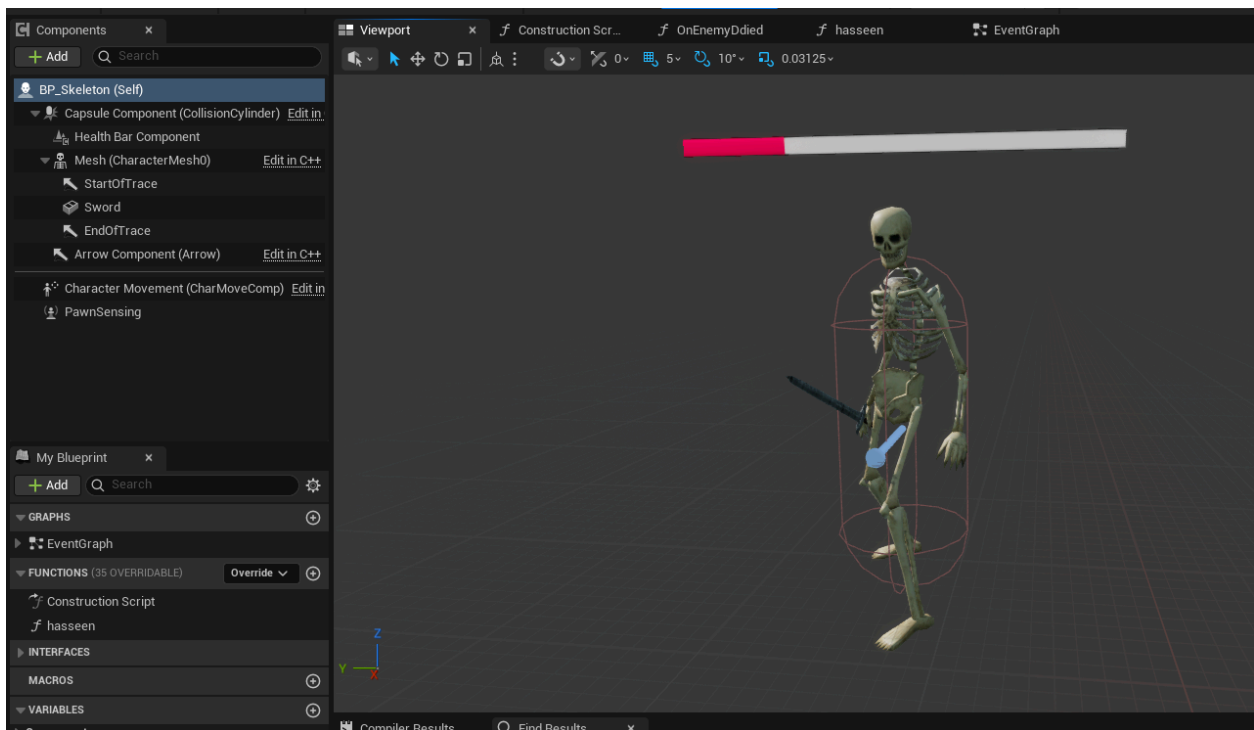


Figura 1: Blueprint perteneciente al esqueleto normal

5.14.1.1. Comportamiento general

Durante el juego (`ReceiveTick`), la barra de vida rota constantemente para orientarse hacia la cámara del jugador, de modo que siempre se muestre de frente independientemente de la posición del enemigo.

Al iniciarse (`ReceiveBeginPlay`), el esqueleto inicializa su barra de vida al 100% y almacena su posición de aparición en la *Blackboard key spawnPoint*, la cual utiliza posteriormente para regresar a su punto de origen.

La detección del jugador se gestiona mediante el evento `On See Pawn` del componente `PawnSensing`. Al detectar al jugador, el esqueleto invoca una función auxiliar (`hasseen`), la cual castea el Pawn detectado a `BP_ThirdPersonCharacter` —validando que corresponda efectivamente al jugador— y, de ser así, marca la *Blackboard key hasSeenPlayer?* como `true`. Esta misma función auxiliar es invocada también desde el evento de recepción de daño, descrito a continuación.

Adicionalmente, al detectar al jugador se inicia un temporizador en bucle de 0.5 segundos que dispara el evento `REGISTER_DISTANCE`, el cual obtiene la referencia al `SlimeGameInstance` e invoca `RegisterDistance`, pasando como parámetro la distancia actual entre el esqueleto y el jugador. Este mecanismo —compartido con el esqueleto caballero, descrito más adelante— es el que alimenta las variables de distancia de la sección de métricas y telemetría durante la fase previa al combate contra el jefe.

5.14.1.2. Recepción de daño y muerte

El procesamiento de daño se centraliza en el evento `ReceiveAnyDamage`. Al recibir daño, en primer lugar se invoca `hasseen` sobre el causante del daño, de modo que un golpe del jugador también provoca que el esqueleto lo detecte (incluso si no lo había visto previamente). A continuación, se resta el daño recibido a la variable `health`, y se actualiza el porcentaje de la barra de vida correspondiente (`health / maxHealth`).

Si la vida resultante es menor o igual a cero, el esqueleto reproduce un sonido de muerte (`SkeleDeath`) y se destruye. Adicionalmente, se obtiene la referencia al jugador (`BP_ThirdPersonCharacter`) y se invoca sobre él la función `RemoveWidget`, encargada de eliminar el indicador visual asociado a este enemigo (por ejemplo, el del sistema de *Target Lock*, en caso de que estuviera fijado sobre él al momento de morir).

Internamente, `RemoveWidget` limpia la variable `Target Lock` del jugador y obtiene la referencia al widget asociado (`TargetLockWidget`), al cual destruye mediante `K2_DestroyActor`. A continuación, restablece las variables de movimiento del jugador (`bOrientRotationToMovement` a `true` y `bUseControllerDesiredRotation`), revirtiendo el cambio de orientación aplicado mientras el enemigo estaba fijado como objetivo, y finalmente destruye el componente del widget (`K2_DestroyComponent`).

5.14.1.3. Ataque

Al igual que en el ataque del jugador, el inicio y fin de la ventana de detección de impacto están delimitados por un *Anim Notify State* anclado al montage de ataque, el cual invoca los eventos `BegomSwordTrace` y `EndSwordTrace` al comenzar y terminar dicha ventana respectivamente.

El ataque del esqueleto utiliza el mismo patrón de detección por temporizador que el ataque del jugador, aunque con una diferencia en su mecanismo de corte. Al iniciar el ataque (*BegomSwordTrace*)¹⁰, se reproduce el sonido de ataque (*SFX_SwordSkele*) y se inicia un temporizador en bucle cada 0.01 segundos que ejecuta el evento *Damage Trace*. Este evento realiza un *Sphere Trace* de radio 30 entre los componentes *StartOfTrace* y *EndOfTrace*; si detecta una colisión válida, aplica el daño definido en la variable *skeleton_damage* sobre el actor impactado y, a diferencia del ataque del jugador, detiene inmediatamente el temporizador mediante *K2_ClearAndInvalidateTimerHandle*, deteniendo la detección tras el primer impacto exitoso. Al finalizar la animación de ataque (*EndSwordTrace*), se invalida el temporizador si aún se encontraba activo, como medida de seguridad en caso de que el ataque no haya conectado con el jugador.

5.14.1.4. Behaviour Tree

El Behaviour Tree (árbol de comportamiento) es el sistema que se encarga de controlar todas las decisiones y acciones de un enemigo en el juego. Funciona evaluando constantemente la situación y eligiendo qué comportamiento ejecutar según la distancia del jugador y otras condiciones. La estructura se organiza en niveles jerárquicos donde cada nivel toma decisiones más específicas que el anterior.

El esqueleto utiliza un Behaviour Tree (*BT_AI*) con una estructura simple de tipo Selector, que evalúa dos secuencias según el estado de la *Blackboard key hasSeenPlayer?*:

- Si *hasSeenPlayer?* no está activa, se ejecuta la secuencia «Look Around»: el esqueleto deambula por el área cercana a su punto de aparición (*BTTask_RoamAround*) y espera 2 segundos antes de repetir el ciclo.
- Si *hasSeenPlayer?* está activa, se ejecuta la secuencia «Chase Player»: el esqueleto persigue al jugador hasta alcanzar la distancia de ataque (*BTTask_ChaseB4Attack*) y luego ejecuta su ataque con espada (*BT_TaskSWORDAttack*). El detalle de cada una de estas tareas se describe a continuación.

El Blackboard es una estructura de datos compartida utilizada por el sistema de inteligencia artificial de Unreal Engine para almacenar información que puede ser accedida y modificada tanto por el Behaviour Tree como por el AI Controller. Funciona como una pizarra donde se escriben y leen valores que representan el estado actual de la IA. Este sistema permite que diferentes componentes de la IA trabajen con la misma información sin necesidad de comunicarse directamente entre sí.

El Blackboard asociado (*BBD_AI*) contiene las siguientes *keys*:

1. *SelfActor* (Object): referencia al propio enemigo.

¹⁰Tal como aparece en el Blueprint; se mantiene el nombre original a pesar de la inconsistencia de escritura respecto a *EndSwordTrace*.

2. `hasSeenPlayer?` (Bool): indica si el enemigo ha detectado al jugador.
3. `IsDead?` (Bool): indica si el enemigo está muerto.
4. `spawnPoint` (Vector): posición de origen, utilizada como punto de patrulla.
5. `distanceToPlayer` (Float): distancia actual respecto al jugador.

5.14.1.5. Tareas del Behaviour Tree

- `BTTask_RoamAround` selecciona, mediante `Get Random Reachable Point In Radius`, un punto alcanzable aleatorio dentro de un radio de 500 unidades alrededor de la posición guardada en `spawnPoint`, y mueve al esqueleto hacia dicho punto con un radio de aceptación de 120 unidades.
- `BTTask_ChaseB4Attack` mueve al enemigo hacia el jugador hasta alcanzar una distancia determinada por la variable `Acceptance Radius`, finalizando la tarea al llegar o al fallar el movimiento. Esta tarea es compartida con el esqueleto mago, el cual altera su comportamiento mediante la variable `infiniteRange` (descrito en la sección correspondiente a dicho enemigo); el esqueleto normal no la utiliza.
- `BT_TaskSWORDAttack` gestiona el ataque cuerpo a cuerpo del esqueleto en dos partes. Al activarse la tarea (`ReceiveExecuteAI`), marca la variable interna `onRotate` como `true` y reproduce el montage de ataque (`MO_Attack`) sobre el esqueleto; en el instante en que la animación lo indica (*Animation Notify*), `onRotate` se marca nuevamente como `false`. Por otro lado, mientras la tarea está activa (`ReceiveTickAI`), si `onRotate` es `true`, el esqueleto rota suavemente hacia la posición del jugador en cada fotograma, interpolando su rotación actual hacia la calculada. De esta forma, el esqueleto se orienta hacia el jugador mientras se prepara para golpear, pero deja de hacerlo justo en el instante del impacto, evitando que el golpe se desvíe por una rotación a mitad de la animación. Al completarse el montage, la tarea finaliza exitosamente.

5.14.2. Esqueleto mago

El esqueleto mago (`BP_SkeletonMage`) comparte la base estructural y gran parte de la lógica del esqueleto normal, descrita en la sección anterior, por lo que a continuación solo se detallan las diferencias respecto a dicho enemigo.

5.14.2.1. Estructura y detección

La jerarquía de componentes es equivalente a la del esqueleto normal: cápsula de colisión con barra de vida, malla esquelética con componentes de escena para el trace de impacto cuerpo a cuerpo (`StartTrace/EndTrace`), componente de movimiento y `PawnSensing`.

La detección del jugador, sin embargo, incorpora una verificación adicional. Al activarse el evento `On See Pawn`, antes de marcar al jugador como detectado se comprueba que la diferencia de altura (eje Z) entre el mago y el jugador no supere las 300 unidades. Esta verificación evita que el mago detecte y reaccione al jugador cuando este se encuentra en un piso o nivel de altura distinto.

A diferencia del esqueleto normal, el mago no reproduce un sonido al morir.

5.14.2.2. Behaviour Tree

El Behaviour Tree del mago reutiliza la secuencia «Look Around» del esqueleto normal (BTTask_RoamAround + espera de 2 segundos) cuando no ha detectado al jugador. Una vez detectado, en lugar de una única secuencia de persecución y ataque, el mago evalúa un segundo Selector que distingue dos comportamientos según la distancia al jugador, mantenida actualizada mediante un servicio (BTS_MageUpdt) que corre en paralelo a ambas secuencias y que, en cada fotograma, calcula la distancia entre el mago y el jugador y la almacena en la *Blackboard key distanceToPlayer*:

- **Secuencia «FarSeq»** (distancia mayor a 280 unidades): el mago se acerca al jugador mediante BTTask_ChaseB4Attack —la misma tarea utilizada por el esqueleto normal—, pero con la variable *infiniteRange* activada. Esto provoca que el AIMoveTo subyacente reciba un radio de aceptación extremadamente amplio, por lo que la tarea se da por completada de inmediato sin que el mago efectivamente se desplace hacia el jugador; en la práctica, el mago no persigue al jugador en esta secuencia. A continuación, ejecuta MagicAttack, descrito más abajo.
- **Secuencia «CloseSeq»** (distancia menor o igual a 280 unidades): el mago ejecuta directamente CloseMage, su ataque cuerpo a cuerpo, descrito más abajo.

La decisión entre ambas secuencias se evalúa mediante la tarea Task_MageDist, la cual comprueba si la distancia actual al jugador (*Blackboard key distanceToPlayer*) se encuentra dentro de un rango definido por las variables *min_dist* y *max_dist* propias de cada secuencia, finalizando con éxito o fracaso según corresponda.

5.14.2.3. Ataque a distancia (MagicAttack)

Al activarse esta tarea, se marca la variable *onRotate* como *true* y se reproduce el montaje M0_MagicAttack. Mientras *onRotate* es *true* (evaluado en cada fotograma, en ReceiveTickAI), el mago interpola su rotación hacia la posición del jugador, de forma idéntica al sistema de orientación visto en el ataque del esqueleto normal. En el instante indicado por un *Animation Notify* del montaje, *onRotate* se marca como *false* —deteniendo la rotación— y se instancia un proyectil (BP_SkeletonBall) desde una posición desplazada 300 unidades hacia adelante respecto al mago, con este último asignado como su dueño (*Owner*). El proyectil se configura con una velocidad de 800, sin influencia de gravedad, sin comportamiento de persecución (*Homing* desactivado), y un daño base de 15. Al completarse el montaje, la tarea finaliza exitosamente.

5.14.2.4. Ataque cuerpo a cuerpo (CloseMage)

Cuando el jugador se encuentra a corta distancia, el mago recurre a un ataque cuerpo a cuerpo con su cayado. La tarea CloseMage marca *onRotate* como *true*, reproduce el montaje M0_CloseStaff, y rota hacia el jugador mediante el mismo mecanismo de interpolación que el resto de las tareas del mago. Al completarse el montaje, *onRotate* se restablece a *false* y

la tarea finaliza con éxito, siguiendo el mismo patrón de montage-completado-finalización empleado en el resto de las tareas de ataque del juego.

Al igual que en los ataques cuerpo a cuerpo del esqueleto normal, la ventana de detección está delimitada por un *Anim Notify State* en el montage `M0_CloseStaff`, que dispara `BegomSwordTrace` y `EndSwordTrace`. Este mecanismo no aplica al ataque a distancia (`MagicAttack`), cuya detección de impacto recae en el propio proyectil al colisionar.

La detección de impacto de este ataque reutiliza el mismo sistema de *trace* por temporizador descrito para el esqueleto normal (`BegomSwordTrace`, `Damage Trace` y `EndSwordTrace`), empleando los componentes `StartTrace` y `EndTrace` del mago. A diferencia del esqueleto normal, donde el daño se obtiene de una variable (`skeleton_damage`), en el mago el daño del ataque cuerpo a cuerpo está definido como un valor fijo de 11.

5.14.3. Esqueleto caballero

El esqueleto caballero (`BP_KnightSkele`) comparte la estructura de componentes y gran parte del comportamiento base del esqueleto normal —cápsula de colisión con barra de vida, malla esquelética con componentes de escena para el *trace* de impacto (`StartTrace/EndTrace`), componente de movimiento y `PawnSensing`—, por lo que a continuación solo se detallan las diferencias respecto a dicho enemigo. Este enemigo se ubica en el nivel `Lvl_PreBoss`, donde —junto con el esqueleto normal— alimenta la métrica de distancia previa al combate contra el jefe.

5.14.3.1. Detección y registro de distancia

Al igual que el esqueleto normal, la barra de vida rota en cada fotograma (`ReceiveTick`) para orientarse hacia la cámara del jugador.

La detección del jugador mediante el evento `On See Pawn` de `PawnSensing` invoca `hasseen`, igual que en los demás esqueletos, e inicia el mismo temporizador en bucle de 0.5 segundos que dispara el evento `REGISTER_DISTANCE`, descrito en la sección del esqueleto normal. La única diferencia es que, naturalmente, la distancia que se calcula y registra corresponde a la posición del esqueleto caballero respecto al jugador, en lugar de la del esqueleto normal.

5.14.3.2. Recepción de daño y muerte

El procesamiento de daño sigue el mismo patrón general que el esqueleto normal (`hasseen` sobre el causante, resta a `health`, actualización de la barra de vida y destrucción al llegar a cero, con la consecuente llamada a `RemoveWidget` sobre el jugador). A diferencia del esqueleto normal, el caballero no reproduce ningún sonido de muerte, igual que el esqueleto mago.

5.14.3.3. Ataque cuerpo a cuerpo (trace por temporizador)

Como en los demás enemigos melee, la ventana de detección está delimitada por un *Anim Notify State* en cada montage de ataque, que dispara `BegomSwordTrace` y `EndSwordTrace`.

El esqueleto caballero también posee, heredado del mismo patrón del esqueleto normal, un ataque cuerpo a cuerpo basado en *trace* por temporizador (*BegomSwordTrace*, *DamageTrace*, *EndSwordTrace*), idéntico en su funcionamiento: temporizador en bucle de 0.01 segundos, *Sphere Trace* de radio 30 entre *StartTrace* y *EndTrace*, y detención inmediata del temporizador tras el primer impacto exitoso. La única diferencia es el nombre de la variable de daño utilizada, *KnightDamage*, en lugar de *skeleton_damage*.

5.14.3.4. Behaviour Tree

El Behaviour Tree del caballero reutiliza la secuencia «Look Around» (*BTTask_RoamAround* + espera de 2 segundos) cuando no ha detectado al jugador, y la tarea *BTTask_ChaseB4Attack* —sin la variable *infiniteRange*, al igual que el esqueleto normal — para acercarse a él una vez detectado. A diferencia de los esqueletos anteriores, tras la persecución el caballero no ejecuta una única tarea de ataque, sino que entra a un nodo compuesto personalizado llamado «Alternating Selector», con dos ramas: *Attack 1* (tarea *KnightAttack1* seguida de una espera de 1 segundo) y *Attack 2* (tarea *KnightAttack2* seguida de una espera de 1 segundo).

5.14.3.4.1. Selector alternante de ataques

El nodo «Alternating Selector» corresponde a la clase *UBTComposite_RandomSelector*, implementada en C++. A pesar de que el nombre de la clase sugiere una selección aleatoria, su lógica (*GetNextChildHandler*) no es aleatoria: en cada ejecución elige el hijo siguiente al último ejecutado mediante el operador módulo sobre la cantidad de hijos ($((\text{LastExecutedChild} + 1) \% \text{GetChildrenNum}())$), partiendo del hijo 0 la primera vez. Es decir, el nodo alterna de forma determinística entre *Attack 1* y *Attack 2* en cada ciclo, en lugar de elegir entre ellos al azar.

5.14.3.4.2. Tareas *KnightAttack1* y *KnightAttack2*

Ambas tareas comparten una misma estructura. Al activarse (*ReceiveExecuteAI*), se marca *onRotate* como *true* y se guarda una referencia casteada al propio caballero (*SkelePawn*), utilizada luego en el cálculo del *trace* de daño. A continuación se reproduce un *Animation Montage*: *M0_AvanceV3* con velocidad de reproducción 1.0 en *KnightAttack1*, y *M0_A0EKnightV3* con velocidad de reproducción 1.2 en *KnightAttack2*.

Al completarse el montage (*OnCompleted*), la tarea finaliza con éxito (*FinishExecute*). En el instante señalado por un único *Animation Notify* del montage (*OnNotifyBegin*), se marca *onRotate* como *false* —deteniendo la rotación hacia el jugador, igual que en el resto de las tareas de ataque del juego— y, a diferencia del ataque con espada por temporizador, se ejecuta un único *Box Trace* (sin temporizador en bucle) entre dos puntos ubicados 150 y 225 unidades respectivamente delante del caballero, en la dirección de su vector hacia adelante. La forma de la caja del trace difiere entre ambas tareas: en *KnightAttack1* es un cubo de 50×50×50 unidades, mientras que en *KnightAttack2* es una caja de 30×80×50 unidades, más ancha. De detectarse una colisión válida, se aplica daño mediante *ApplyDamage*, con

un valor fijo de 20 en ambas tareas (a diferencia del `KnightDamage` variable utilizado en el ataque con espada).

Mientras la tarea está activa (`ReceiveTickAI`), y siguiendo el mismo patrón visto en el resto de los enemigos, el caballero rota suavemente hacia el jugador mediante `RInterpTo` (con una velocidad de interpolación de 3.0) mientras `onRotate` sea `true`, deteniéndose justo en el instante del impacto.

5.15. Jefe (BP_Slime)

5.15.1. Modelo y animaciones

El modelo y las animaciones del jefe fueron creados desde cero en Blender, en lugar de recurrir a assets externos como en el resto de los enemigos. La motivación principal fue contar con un control total sobre el resultado, de modo que el modelo y sus animaciones pudieran ajustarse específicamente a los requerimientos de *gameplay* del jefe —en particular, a los *Anim Notify States* que delimitan las ventanas de ataque y a las distintas formas que necesita adoptar durante el combate.

Dado que el slime no posee un esqueleto tradicional, gran parte de sus deformaciones se modelaron mediante *shape keys*, tanto para las distintas formas que puede adoptar el cuerpo (por ejemplo, su forma de charco, o su forma donde de espinas) como para las acciones propias de cada ataque.

5.15.2. Componentes y variables

La jerarquía de componentes de `BP_Slime` es la siguiente:

1. `CollisionCylinder`: cápsula de colisión, raíz del actor.
2. `PULL_OUT`: componente de colisión de tipo esfera, utilizado para detectar si el jugador quedó atrapado dentro del jefe tras ciertos ataques (ver Sección 5.15.5.2).
3. `CharacterMesh0`: malla esquelética del jefe.
 1. `StartTracePos`: componente de escena, punto de inicio del *trace* de impacto cuerpo a cuerpo.
 2. `EndTracePos`: componente de escena, punto de fin del *trace* de impacto cuerpo a cuerpo.
 3. `Arrow`: componente de flecha, referencia direccional.
4. `PlayerMetrics (PlayerMetricsComponent)`: descrito en Sección 5.16.
5. `CharMoveComp`: componente de movimiento de personaje.

Adicionalmente, `BP_Slime` implementa la interfaz `BPI_Lockable`, consultada por el sistema de fijado de objetivo del jugador (*Target Lock*, descrito en la sección correspondiente) para obtener el punto exacto al que debe apuntar la cámara mientras el jefe está fijado. Esto permite compensar el desplazamiento visual que sufre el jefe durante su animación de ataque de charco, donde se hunde visualmente sin que ese desplazamiento se traduzca en un

movimiento real del actor. Su función `Get Lock On Target` retorna una posición desplazada hacia abajo respecto a la ubicación del actor: 300 unidades en el eje Z si `bIsCharco` es `true`, o 100 unidades si es `false`.

Las variables propias del Blueprint son:

1. `health` (Float): vida actual del jefe.
2. `max_health` (Float): vida máxima del jefe.
3. `progressbarref` (Progress Bar): referencia al widget de la barra de vida.
4. `bIsCharco` (Bool): indica si el jefe se encuentra en su forma de charco.

5.15.3. AI Controller

El AI Controller del jefe (`BP_SlimeBossController`) expone una función llamada `ActivateBossController`, invocada desde una zona de activación (*trigger*) al inicio de la arena del jefe. Al ejecutarse, esta función arranca el Behaviour Tree (`BT_BaseSlimeBoss`), registra la referencia al jugador en el Blackboard (`PlayerActor`) e invoca `ApplyPreCombatAdaptation` sobre el `PlayerMetricsComponent`, aplicando los ajustes de pesos de ataque derivados de las métricas recolectadas durante la fase previa al combate. Finalmente, vuelca el estado resultante de los pesos a archivo mediante `DumpFinalTreeToFile`, tal como se describió en la sección Sección 5.16.

5.15.4. Blackboard

El Blackboard del jefe (`BBD_SlimeBoss`) contiene las siguientes *keys*:

1. `SelfActor`: Tipo *Actor*. Referencia al propio actor controlado por la IA, utilizada para acceder a sus datos y ejecutar acciones sobre sí mismo.
2. `PlayerActor`: Tipo *Actor*. Referencia al jugador detectado por la IA, empleada para seguimiento, persecución y ataques.
3. `PlayerDistance`: Tipo *Float*. Almacena la distancia actual entre la IA y el jugador.
4. `LastAttack`: Tipo *Enum* (`EBossAttackType`). Guarda el último ataque ejecutado por el jefe.
5. `LastAttackCount`: Tipo *Integer*. Cuenta cuántas veces consecutivas se ha seleccionado el mismo ataque almacenado en `LastAttack`.
6. `AttackWeightsJSON`: Tipo *String*. Contiene datos de configuración de pesos de ataques almacenados en formato JSON.
7. `CloseRange`: Tipo *Float*. Distancia considerada como rango cercano para la toma de decisiones de combate.
8. `FarRange`: Tipo *Float*. Distancia considerada como rango lejano para la toma de decisiones de combate.
9. `0-KEY`: Tipo *Float/Integer*. Valor asociado a la clave «0» dentro de una estructura de configuración o tabla de pesos.
10. `FAR-KEY`: Tipo *Float/Integer*. Valor asociado a la clave de comportamiento o ataque de larga distancia.

11. **BossChaseDur**: Tipo *Float*. Duración durante la cual el jefe mantiene el estado de persecución antes de cambiar de comportamiento.

5.15.5. Behaviour Tree

El Behaviour Tree del jefe se organiza en niveles jerárquicos. El nodo raíz conecta con un Selector principal que evalúa tres secuencias según la distancia al jugador (*Blackboard key* `PlayerDistance`, actualizada por el service `BTS_UPDTPlayerDistance` descrito más arriba), intentando cada una en orden hasta que alguna tiene éxito:

1. **Secuencia lejana**: distancia mayor a `FarRange`.
2. **Secuencia media**: distancia entre `CloseRange` y `FarRange`.
3. **Secuencia cercana**: distancia menor a `CloseRange`.

Dentro de cada secuencia, un nodo `Task_SelectDistance` confirma que la distancia actual cumple el rango correspondiente y, a continuación, un nodo `BTComposite_RandomSelector`—descrito en la siguiente sección, con una instancia por cada secuencia— reparte mediante selección aleatoria ponderada entre un subconjunto de los nueve ataques del jefe:

1. Secuencia lejana: Charco (`BA_Poddle`), Persecución (`BA_BossChase`), proyectil en línea recta (`BA_ProjectilAttack`) y proyectil homing (`BA_HomingAttack`).
2. Secuencia media: Básico (`BA_BasicAttack`), salto (`BA_HeavyAttack`) y giro (`BA_WhipAttack`).
3. Secuencia cercana: Básico (`BA_BasicAttack`), espinas (`BA_A0EAttack`) y muro (`BA_WallAttack`).

5.15.5.1. BTComposite_RandomSelector (C++)

La elección del ataque dentro de cada secuencia de rango se gestiona mediante un nodo compuesto personalizado implementado en C++ (`UCustomCompositeNode`, mostrado en el editor como «RandomSelectWeight»). A diferencia de un Selector convencional, que recorre sus hijos en orden hasta que uno tiene éxito, este nodo elige un único hijo mediante una selección aleatoria ponderada y, una vez que dicho hijo termina de ejecutarse, retorna directamente a su padre (`GetNextChildHandler` siempre devuelve `ReturnToParent` tras la primera ejecución), sin intentar el resto de sus hijos.

La selección (`ChooseWeightedChild`) asigna a cada hijo un peso base de 1.0, el cual es sobrescrito si existe una entrada con su nombre dentro del JSON almacenado en la *Blackboard key* `AttackWeightsJSON` (ver Sección 5.16.6). Sobre estos pesos se aplica además una regla anti-repetición: si el último ataque ejecutado (`LastAttack`) se repitió dos veces consecutivas (`LastAttackCount` $\geq$ 2), el peso del hijo correspondiente a ese mismo ataque se fuerza a 0, impidiendo que se seleccione una tercera vez consecutiva. Sobre el arreglo de pesos resultante se realiza un sorteo aleatorio ponderado para elegir el hijo a ejecutar.

Tras la selección, se actualiza el Blackboard: si el ataque elegido es el mismo que el anterior, se incrementa `LastAttackCount`; en caso contrario, `LastAttack` se actualiza al nuevo ataque y `LastAttackCount` se reinicia a 1.

En la raíz del Behaviour Tree corre un service (`BTS_UPDTPlayerDistance`), análogo al `BTS_MageUpdt` del esqueleto mago, que en cada fotograma calcula la distancia entre el jefe y el jugador y la almacena en la *Blackboard key* `PlayerDistance`.

5.15.5.2. Tareas de ataque

El jefe cuenta con nueve tareas de ataque distintas, identificadas mediante el enum `EBossAttackType` (`BA_BasicAttack`, `BA_AOEAttack`, `BA_HeavyAttack`, `BA_ProjectilAttack`, `BA_BossChase`, `BA_Poddle`, `BA_HomingAttack`, `BA_WhipAttack`, `BA_WallAttack`), seleccionadas por el `BTComposite_RandomSelector` descrito en la sección anterior.

5.15.5.2.1. Ataque básico (`BA_BasicAttack`)

Al activarse la tarea (`ReceiveExecuteAI`), se registra el intento de ataque (`RegisterBossAttackAttempt`, con tipo `BA_BasicAttack`) y se reproduce el montaje `M0_FixBasicAttack`, deteniendo cualquier otro montaje en reproducción (`bShouldStopAllMontages`). Al completarse o interrumpirse el montaje, la tarea finaliza con éxito. En el instante señalado por un único *Animation Notify*, se ejecuta un único *Box Trace* (sin temporizador en bucle) sobre una caja estática de $100 \times 100 \times 100$ unidades, centrada 200 unidades hacia adelante de la posición del jefe y orientada según su rotación; de detectarse una colisión válida, se aplica un daño fijo de 10.

5.15.5.2.2. Ataque de área (`BA_AOEAttack`)

Al activarse, se registra el intento de ataque y se reproduce el montaje `M0_SpykeV4`. Al completarse o interrumpirse, la tarea finaliza con éxito. En el instante señalado por el *Animation Notify*, se ejecuta un *Sphere Trace* de radio 370 unidades centrado en la posición del jefe; de detectarse una colisión válida, se aplica un daño definido por la variable `SpykeDmg`.

5.15.5.2.3. Ataque pesado (`BA_HeavyAttack`)

El ataque pesado corresponde a un ataque con salto. Al activarse, se registra el intento de ataque y se almacenan, como instantánea (*snapshot*, sin actualizarse posteriormente), la posición inicial del jefe y la posición del jugador en ese momento. El jefe pasa a modo de movimiento `Flying` y deja de colisionar con `Pawns`, y reproduce el montaje `M0_JumpAttackV3`. Transcurridos 0.9 segundos desde el inicio del montaje, se marca la variable `bMove` como `true`; mientras esta sea `true`, en cada fotograma (`ReceiveTickAI`) el jefe se desplaza hacia la posición guardada del jugador y rota hacia ella. En el instante señalado por el *Animation Notify*, se invoca la función `Hitbox&KnockBack`.

Al completarse o interrumpirse el montaje, el jefe vuelve a modo `Walking`, restaura su colisión con `Pawns` y, si completó normalmente, comprueba si quedó atrapado contra algún

actor mediante el componente `PULL_OUT` —de la misma forma descrita para el ataque de charco (`BA_Poddle`), más abajo— antes de finalizar la tarea con éxito.

5.15.5.2.4. Ataque de proyectil (`BA_ProjectilAttack`)

Al activarse, se reproduce el montaje `M0_ProjectileAttack`. Al completarse o interrumpirse, la tarea finaliza con éxito. En el instante señalado por el *Animation Notify*, se recorre un arreglo fijo de desviaciones angulares (0° , 15° , -15° , 30° , -30° , 45° y -45°) y, por cada una, se instancia un proyectil (`BP_BossSlimeBall_C`) 100 unidades por debajo de la posición del jefe, con su rotación compuesta a partir de la desviación correspondiente y la rotación del jefe, generando un disparo en abanico de siete proyectiles. Cada proyectil se configura con el jefe como dueño (*Owner*), velocidad 1000, sin gravedad, sin comportamiento de persecución (*Homing* desactivado) y un daño base de 10. Mientras la tarea está activa (`ReceiveTickAI`), el jefe rota hacia el jugador con una velocidad de interpolación de 0.5.

A diferencia del resto de las tareas de ataque, ni esta ni el ataque homing (`BA_HomingAttack`, descrito más abajo) invocan `RegisterBossAttackAttempt`: al tratarse de ataques a distancia, no es directo determinar si efectivamente conectaron con el jugador, por lo que no se registran en las métricas de ataque.

5.15.5.2.5. Persecución (`BA_BossChase`)

Al activarse, esta tarea inicia un `AIMoveTo` hacia el jugador (`TargetActor`), con un radio de aceptación propio de la tarea (`Acceptance Radius`). En paralelo, corre un `Delay` de una duración también propia de la tarea (`Duration`), que actúa como límite de tiempo. Si el `Delay` se completa primero, se detiene el movimiento y la tarea finaliza con éxito. Si el `AIMoveTo` tiene éxito primero, se ejecuta ese mismo bloque de detención de movimiento y finalización.

5.15.5.2.6. Charco (`BA_Poddle`)

Esta tarea hace uso de la forma de charco del jefe, modelada mediante *shape keys* (ver Sección 5.15.1). Al activarse, se registra el intento de ataque, se ajusta la velocidad de movimiento a 600 y se desactiva la colisión del jefe contra `Pawns` y contra dos canales de colisión adicionales del proyecto. Se reproduce el montaje `M0_PoddleStart`, se inicia un `AIMoveTo` hacia el jugador (sin radio de aceptación), y se programan dos temporizadores: uno a 4 segundos (`StartDetecting`) y otro a 7 segundos (`TimeoutdeAtaque`). Finalmente, se marca `bIsCharco` como `true` y se reproduce el montaje `M0_PoddleDown`, quedando el jefe en su forma de charco.

Al dispararse `StartDetecting` (4 segundos después de iniciada la tarea), se marca `bIsDetectingPlayer` como `true`. A partir de ese momento, en cada fotograma (`ReceiveTickAI`) se comprueba, mientras `bIsDetectingPlayer` sea `true`, si el jugador se encuentra a menos de 250 unidades del jefe; de ser así, se marca `bIsDetectingPlayer` nuevamente como `false`, se cancela el temporizador de `TimeoutdeAtaque` y se invoca dicho

evento manualmente, adelantando el fin del ataque a que el jugador se haya acercado lo suficiente.

TimeoutdeAtaque —ya sea disparado por su propio temporizador a los 7 segundos, o adelantado desde el **Tick** como se describió arriba— detiene el movimiento del jefe, reduce su velocidad a 285 y reproduce el montage **MO_PoddleUp**. En el instante señalado por el *Animation Notify* de este montage, se ejecuta un *Sphere Trace* de radio 225 alrededor del jefe; de detectarse al jugador, se le aplica un daño definido por la variable **PoddleDamage**, se cancelan sus montages de esquivar en las cuatro direcciones, y se lo lanza por los aires (*launch*). Al completarse el montage, se restaura la colisión del jefe (bloqueo contra Pawns, bloqueo contra uno de los canales adicionales y *overlap* contra el otro) y se marca **bIsCharco** nuevamente como **false**.

Finalmente, se comprueba si el jefe quedó atrapado dentro de otro actor durante la transformación: se obtienen los actores que se superponen con el componente **PULL_OUT** y se evalúa si dicho arreglo está vacío. Si está vacío, la tarea finaliza con éxito directamente. Si no lo está, el jefe se reposiciona, desplazándose en la dirección normalizada desde el actor encontrado hacia el jefe (a 1000 unidades de distancia, con la altura igualada a la del jugador), y la tarea finaliza con éxito a continuación.

5.15.5.2.7. Ataque homing (BA_HomingAttack)

Esta tarea es idéntica en estructura al ataque de proyectil (**BA_ProjectileAttack**, descrito más arriba): reproduce el mismo montage (**MO_ProjectileAttack**) y, en su *Animation Notify*, instancia proyectiles (**BP_BossSlimeBall_C**) en abanico. Las diferencias son el arreglo de desviaciones angulares utilizado —0°, 60° y −60°, es decir, tres proyectiles en lugar de siete—, la velocidad de los proyectiles (850 en vez de 1000) y que estos sí tienen activado el comportamiento de persecución (*Homing*). Al igual que el ataque de proyectil normal, esta tarea no invoca **RegisterBossAttackAttempt**.

5.15.5.2.8. Ataque de látigo (BA_WhipAttack)

El ataque de látigo corresponde a un ataque tipo espada/látigo. Al activarse, se registra el intento de ataque y se reproduce el montage **MO_SwordSlimeAttack**. Mientras la tarea está activa (**ReceiveTickAI**), el jefe rota hacia el jugador con una velocidad de interpolación de 1.0.

A diferencia del resto de los ataques del jefe, la detección de impacto de este ataque no se resuelve mediante un trace dentro de esta misma tarea, sino a través de una interfaz de notificación de daño aparte.

5.15.5.2.9. Ataque de muro (BA_WallAttack)

Al activarse, se marca la variable **canRotate** como **true**, se registra el intento de ataque y se reproduce el montage **MO_WallAttack**. En el instante señalado por el *Animation Notify*, se marca **canRotate** como **false** —deteniendo la rotación hacia el jugador— y se ejecuta

un único *Box Trace* entre la posición del jefe y un punto 520 unidades hacia adelante, con una caja de 100×100×100 unidades orientada según su rotación; de detectarse una colisión válida, se aplica un daño fijo de 10. Al completarse el montaje, la tarea finaliza con éxito. Mientras la tarea está activa y `canRotate` sea `true`, el jefe rota hacia el jugador en cada fotograma con una velocidad de interpolación de 1.0.

5.15.6. PlayerMetricsComponent

El jefe cuenta con un componente `PlayerMetricsComponent`, encargado de registrar la efectividad de sus ataques durante el combate y de traducir las métricas recolectadas en ajustes sobre su propio comportamiento (los pesos de selección de ataque descritos en Sección 5.16.6). Su funcionamiento se detalla en profundidad en Sección 5.16, sección transversal a varios sistemas del juego.

5.15.7. Secuencia de introducción

5.16. Sistema de métricas y telemetría

De forma transversal a varios de los sistemas descritos en este capítulo, el juego incorpora una infraestructura de recolección y uso de métricas sobre el comportamiento del jugador, implementada principalmente en dos clases de C++: `SlimeGameInstance`, encargada de recolectar las métricas y persistirlas, y `PlayerMetricsComponent`, anclado al jefe, encargado de registrar la efectividad de sus ataques y de traducir las métricas recolectadas en ajustes sobre su comportamiento.

5.16.1. Variables de SlimeGameInstance

Como se mencionó, `SlimeGameInstance` recolecta sus datos en dos fases paralelas, controladas mediante la variable `isInCombat`: una fase previa al combate (en `Lvl_PreBoss`), condicionada además a que `bMetricsEnabled` esté activo, y una fase durante el combate contra el jefe, cuyas variables llevan el prefijo `Combat`. A continuación se detallan las variables principales, agrupadas por categoría.

Cabe precisar de antemano el rol efectivo de estas variables con prefijo `Combat` dentro del sistema de adaptación. Si bien se registran de forma paralela a sus contrapartes sin prefijo —mediante el mismo flag `isInCombat`— y forman parte del diseño original de la infraestructura de métricas, que contemplaba adaptar también el comportamiento del jefe durante el combate en base a estas métricas, dicha adaptación en tiempo real no se llegó a implementar. La adaptación pre-combate (`ApplyPreCombatAdaptation` y sus subfunciones, ver Sección 5.16.6) lee exclusivamente las variables sin prefijo; ningún punto del sistema consulta las variables `Combat*` para tomar decisiones. Las únicas excepciones, es decir, las únicas métricas recolectadas durante el combate que sí tienen un efecto real sobre el comportamiento del jefe, son el registro de aciertos por tipo de ataque (`TotalAttemptsPerType` y `SuccessfulHits`, en `PlayerMetricsComponent`), que alimenta `ApplyInCombatSuccessfulHits`, y el monitoreo de curación en combate vía `ApplyInCombatHealing`, que compara la vida actual del jugador contra `AverageHealingHP` —calculada en la fase pre-combate— para ajustar la presión de ataque cuando el jugador entra en su rango habitual de curación.

Distancia

1. **DistanceAccum, DistanceSamples**: acumulador y contador de muestras de distancia (pre-combate).
2. **AverageDistance**: promedio de distancia, recalculado en cada muestra (pre-combate).
3. **CombatDistanceAccum, CombatDistanceSamples, CombatAverageDistance**: equivalentes durante el combate.

Ataques

1. **MeleeAttacks, RangedAttacks**: conteo de ataques melee y a distancia (pre-combate).
2. **CombatMeleeAttacks, CombatRangedAttacks**: equivalentes durante el combate.
3. **MeleeAttacksInRangeZone, RangedAttacksInRangeZone**: conteo de cada tipo de ataque realizado dentro de una zona de rango.

Curación

1. **HealsAtHighHP, HealsAtMidHP, HealsAtLowHP**: conteo de curaciones según el rango de vida del jugador al curarse (pre-combate).
2. **HealingHPAccum, HealingCount**: acumulador y contador para el promedio de vida al curarse.
3. **AverageHealingHP**: promedio del porcentaje de vida del jugador al momento de curarse.
4. **CombatHealsAtHighHP, CombatHealsAtMidHP, CombatHealsAtLowHP, CombatHealingHPAccum, CombatHealingCount, CombatAverageHealingHP**: equivalentes durante el combate.

Esquivas y daño

1. **TotalDodges, TotalDodgesTankZone**: conteo total de esquivas, y de aquellas realizadas en la zona de tanque (pre-combate).
2. **SuccessfulDodges**: conteo de esquivas marcadas explícitamente como exitosas.
3. **DamageTaken, TotalDamageReceived**: conteo de impactos recibidos y daño total acumulado (pre-combate).
4. **CombatTotalDodges, CombatSuccessfulDodges**: equivalentes durante el combate.
5. **CombatDamageTaken, CombatTotalDamageReceived**: equivalentes durante el combate.
6. **LateralDodgesFromDash, BackwardDodgesFromDash**: esquivas laterales o hacia atrás frente a el ataque de embestida del caballero esqueleto.
7. **DodgesFromDelayAttack, DodgesFromDelayAttack_TankZone**: esquivas frente a un ataque con retardo, en general y en zona de tanque.
8. **CombatDodgesFromDelayAttack**: equivalente durante el combate.

Zona de tanque

1. **isInTankZone**: flag de si el jugador se encuentra actualmente en la zona de tanque.
2. **DamageTakenInTankZone**: conteo de daño recibido en dicha zona.

Estudio

1. **bAdaptiveEnabled**: condición experimental del jugador (`true` = adaptativo, `false` = control).
2. **bMetricsEnabled**: habilita o no la recolección durante el nivel.
3. **SessionId**: identificador de sesión, generado a partir de fecha y hora.
4. **AttemptNumber**: número de intento actual contra el jefe.

5.16.2. Funciones de registro (**SlimeGameInstance**)

Sobre las variables anteriores operan las siguientes funciones, todas siguiendo el mismo patrón: bifurcan su comportamiento según `isInCombat`, actualizando el conjunto de variables de la fase correspondiente.

1. **RegisterDistance(Distance)**: acumula `Distance` en `DistanceAccum` (o su equivalente de combate), incrementa el contador de muestras, y recalcula el promedio dividiendo el acumulador entre el número de muestras.
2. **RegisterMeleeAttack()** y **RegisterRangeAttack()**: incrementan `MeleeAttacks` o `RangedAttacks` respectivamente (o sus equivalentes de combate); adicionalmente, si `isInRangedZone` es verdadero, incrementan también `MeleeAttacksInRangedZone` o `RangedAttacksInRangedZone` según corresponda.
3. **RegisterHealingMoment(PlayerHealth, PlayerMaxHealth)**: calcula $(\text{PlayerHealth} / \text{PlayerMaxHealth}) * 100$, incrementa `HealingCount` y acumula el porcentaje en `HealingHPAccum` para recalcular `AverageHealingHP`; según el porcentaje resultante, incrementa `HealsAtHighHP` ($> 70\%$), `HealsAtMidHP` ($> 30\%$) o `HealsAtLowHP` ($\leq 30\%$).
4. **RegisterTotalDodge()**: incrementa `TotalDodges`; si `isInTankZone` es verdadero, incrementa además `TotalDodgesTankZone`.
5. **RegisterDodgeResult(bWasSuccessful)**: si `bWasSuccessful` es verdadero, incrementa `SuccessfulDodges`. Si es falso, la función no realiza ninguna acción.
6. **RegisterDamageTaken(Damage)**: incrementa `DamageTaken` y acumula `Damage` en `TotalDamageReceived`.
7. **RegisterDashDodge(bWasLateral)**: incrementa `LateralDodgesFromDash` o `BackwardDodgesFromDash` según `bWasLateral`. A diferencia de las anteriores, no bifurca por `isInCombat`; solo depende de `bMetricsEnabled`.
8. **RegisterDodgeFromDelayAttack()**: incrementa `DodgesFromDelayAttack` (o `CombatDodgesFromDelayAttack` en combate); si `isInTankZone` es verdadero, incrementa además `DodgesFromDelayAttack_TankZone` en ambos casos.
9. **RegisterDamageTakenInTankZone()**: incrementa `DamageTakenInTankZone`. No bifurca por `isInCombat`, solo depende de `bMetricsEnabled`.

5.16.3. Registro de ataques del jefe (**PlayerMetricsComponent**)

`PlayerMetricsComponent` lleva un registro independiente mediante dos mapas indexados por tipo de ataque (`EBossAttackType`): `TotalAttemptsPerType` y `SuccessfulHits`.

`RegisterBossAttackAttempt(AttackType)` incrementa el contador de intentos correspondiente en `TotalAttemptsPerType`, además de un contador global `TotalAttacksPerformed`.

Cada 15 ataques realizados (`TotalAttacksPerformed % 15 == 0`), se dispara automáticamente `ApplyInCombatSuccessfulHits`.

`RegisterBossAttackHit(AttackType)` incrementa el contador de aciertos correspondiente en `SuccessfulHits`.

5.16.4. Adaptación pre-combate

Al finalizar la fase previa al combate, `ApplyPreCombatAdaptation` analiza las variables de `SlimeGameInstance` y ajusta los pesos de selección de ataques del jefe (mecanismo descrito en Sección 5.16.6) en cuatro dimensiones independientes, llamando para ello a las funciones `ApplyPreCombatDistance`, `ApplyPreCombatMeleeVsRanged`, `ApplyPreCombatDodges` y `ApplyPreCombatTankZone`, descritas a continuación. Todo el sistema de adaptación solo se ejecuta si `IsAdaptiveEnabled` retorna verdadero, lo cual depende de `bAdaptiveEnabled`.

Distancia (`ApplyPreCombatDistance`): requiere `DistanceSamples >= 20`. Si `AverageDistance < 550`, reduce en 50 la variable de Blackboard `CloseRange`. Si `AverageDistance > 1100`, reduce en 75 `FarRange` e incrementa en 2 segundos `BossChaseDur`. Entre ambos umbrales, no hay ajuste.

Melee versus a distancia (`ApplyPreCombatMeleeVsRanged`): requiere `MeleeAttacks + RangedAttacks >= 5`. Calcula `RangedRatio = RangedAttacks / Total`. Si `RangedRatio > 0.65`, incrementa en 15 los pesos de `BA_Poddle`, `BA_BossChase` y `BA_HeavyAttack`. Si `RangedRatio < 0.35`, incrementa en 15 los pesos de `BA_AOEAttack` y `BA_WhipAttack`. Adicionalmente, si `MeleeAttacksInRangedZone + RangedAttacksInRangedZone >= 3`, incrementa en 10 el peso de `BA_AOEAttack` si predominó el melee en zona de rango, o el de `BA_Poddle` si predominó el ataque a distancia.

Esquiva (`ApplyPreCombatDodges`): requiere `TotalDodges >= 5`. Calcula `DodgeRatio = DodgesFromDelayAttack / TotalDodges`. Si `DodgeRatio > 0.6`, incrementa en 15 el peso de `BA_BasicAttack`. Si `DodgeRatio < 0.3`, incrementa en 15 los pesos de `BA_AOEAttack`, `BA_WallAttack` y `BA_HeavyAttack`. Entre ambos umbrales, no hay ajuste.

Zona de tanque (`ApplyPreCombatTankZone`): evalúa dos aspectos. Si `LateralDodgesFromDash + BackwardDodgesFromDash >= 3`, incrementa en 10 el peso de `BA_WhipAttack` (si predominaron las laterales) o de `BA_HeavyAttack` (si predominaron las de retroceso). Si `TotalDodgesTankZone >= 3`, compara si `DodgesFromDelayAttack_TankZone * 2 >= TotalDodgesTankZone`: de ser así, incrementa en 10 el peso de `BA_BasicAttack`; en caso contrario, incrementa en 10 los pesos de `BA_HeavyAttack` y `BA_WhipAttack`.

5.16.5. Adaptación durante el combate

`ApplyInCombatSuccessfulHits`, disparada cada 15 ataques del jefe, evalúa cada tipo de ataque con `TotalAttemptsPerType >= 3`, calculando `Ratio = SuccessfulHits /`

`TotalAttemptsPerType`. Si `Ratio > 0.6`, incrementa el peso de ese ataque en 15. Si `Ratio < 0.3`, lo reduce en 15. Entre ambos umbrales, no hay ajuste.

`ApplyInCombatHealing(PlayerHealthPercent)` requiere `HealingCount >= 2`. Compara `PlayerHealthPercent` contra `AverageHealingHP` con un margen de 10 puntos porcentuales. Al entrar en dicho margen (controlado por la variable interna `bIsInHealingRange`, para evitar reaplicar el ajuste en cada frame), incrementa en 15 los pesos de `BA_BasicAttack`, `BA_HeavyAttack`, `BA_HomingAttack` y `BA_Poddle`. Al salir del margen, revierte los mismos ajustes en -15 .

5.16.6. Mecanismo de pesos de ataque

Los pesos de selección de ataques del jefe, utilizados por el nodo compuesto de selección aleatoria ponderada del Behaviour Tree, se almacenan como una única cadena en formato JSON dentro de la variable de Blackboard `AttackWeightsJSON`, en lugar de como variables individuales. `SetAttackWeight(AttackType, NewWeight)` deserializa dicha cadena, actualiza el peso correspondiente al tipo de ataque indicado, y vuelve a serializarla y almacenarla. `GetAttackWeight(AttackType)` realiza el proceso inverso, retornando el peso actual de un tipo de ataque (o -1 si no se encuentra).

5.16.7. Persistencia de datos para el estudio

Los datos se persisten en archivos JSON dentro de `Saved/StudyLogs`:

1. `DumpMetricsToFile (SlimeGameInstance)`: vuelca las variables de la fase pre-combate (`AverageDistance`, `DistanceSamples`, `MeleeAttacks`, `RangedAttacks`, `MeleeAttacksInRangedZone`, `RangedAttacksInRangedZone`, `TotalDodges`, `SuccessfulDodges`, `AverageHealingHP`, `HealingCount`, `LateralDodgesFromDash`, `BackwardDodgesFromDash`, `DodgesFromDelayAttack`, `DamageTakenInTankZone`, `TotalDodgesTankZone`, `DodgesFromDelayAttack_TankZone`) a `session_{SessionId}_{condición}_metrics.json`.
2. `DumpCombatResultToFile(bPlayerWon) (PlayerMetricsComponent)`: vuelca `bPlayerWon`, `AttemptNumber` y el contenido de `AttackWeightsJSON` a `session_{SessionId}_{condición}_win.json` o `_retry_{AttemptNumber}.json`, según corresponda.
3. `DumpFinalTreeToFile (PlayerMetricsComponent)`: vuelca únicamente `AttackWeightsJSON` a `session_{SessionId}_{condición}_tree.json`.

En todos los casos, `SessionId` (generado a partir de fecha y hora) y la condición experimental (`adaptive` o `control`, según `bAdaptiveEnabled`) identifican tanto el contenido como el nombre del archivo.

5.17. Niveles

Los niveles del juego comparten una ambientación de mazmorra, construida a partir de *assets* obtenidos principalmente de una entrega gratuita de Fab (Unreal Engine Marketplace), complementados con *assets* de otras fuentes. La estructura general de los niveles

—cómo se distribuyen y conectan entre sí— se describe en la sección 3.3; a continuación se documentan los elementos reutilizables que los componen y, luego, la implementación particular de cada nivel.

5.17.1. Elementos de nivel

5.17.1.1. Maniquí

5.17.1.2. Puerta (BP_Door)

5.17.1.3. Espinas

5.17.1.4. Puente

5.17.1.5. Palanca

5.17.1.6. Escalera

Corresponde únicamente a un *Static Mesh* de escalera, sin lógica de Blueprint asociada.

5.17.2. Lvl_Tutorial

El nivel de tutorial consiste en un mapa de mazmorra que combina varios de los elementos descritos arriba —un maniquí, puertas (BP_Door), espinas y un puente— junto con un esqueleto normal que ronda libremente por el área. La sala inicial presenta al jugador con un maniquí; al derrotarlo, se abre una puerta que da paso al resto del nivel.

5.17.3. Lvl_PreBoss

El nivel previo al jefe combina palancas, puertas, escaleras (como elemento puramente visual/de navegación) y los tres tipos de enemigo esqueleto descritos anteriormente (esqueleto normal, esqueleto mago y esqueleto caballero).

5.17.4. Lvl_ThirdPerson

5.17.5. Widget de tutorial (WB_PopChico)

El widget WB_PopUpChico muestra los textos de tutorial (controles del juego) como un popup centrado en la parte inferior de la pantalla. Su aparición y desaparición están controladas por zonas delimitadas mediante *Box Collision* distribuidas en el nivel: al entrar a una de estas zonas se muestra el popup, y al salir de ella se oculta. El contenido mostrado depende únicamente de cuál *Box Collision* específica fue la que se activó —cada zona define directamente su propio texto—, sin depender de la dirección o el lado por el que el jugador haya entrado.

5.18. Widgets generales (MainMenu, WinnerScreen...)

5.18.1. Menú principal (WB_MainMenu)

5.18.2. Pantalla de victoria (WB_WinnerScreen)

5.18.3. Barra de vida del jefe (WB_BossHealth)

5.18.4. Barra de vida de enemigos normales (WB_NormalHealth)

Capítulo 6

Prueba de concepto y evaluación

Capítulo 7

Conclusión y trabajo futuro

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliografía

- [1] João Paulo Sousa, Rogério Tavares, João Pedro Gomes, y Vitor Mendonça, «Review and analysis of research on Video Games and Artificial Intelligence: a look back and a step forward», *Procedia Computer Science*, [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S1877050922007761>
- [2] Mohammad Zohaib, «Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review», *Advances in Human-Computer Interaction*, vol. 2018, p. 5681652.
- [3] W. A. IJsselsteijn, Y. A. W. de Kort, y K. Poels, «The Game Experience Questionnaire».
- [4] Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ögren, y Christian Smith, «A survey of Behavior Trees in robotics and AI», *Robotics and Autonomous Systems*, vol. 154, p. 104096, [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0921889022000513>
- [5] Michael Booth, «The AI Systems of Left 4 Dead». [En línea]. Disponible en: <https://www.gdcvault.com/play/1015528/The-AI-Systems-of-Left>
- [6] David Pinelle, Nelson Wong, y Tadeusz Stach, «Heuristic Evaluation for Games: Usability Principles for Video Game Design», en *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2008)*, ACM, pp. 1453-1462.

Anexo A

Anexo

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.